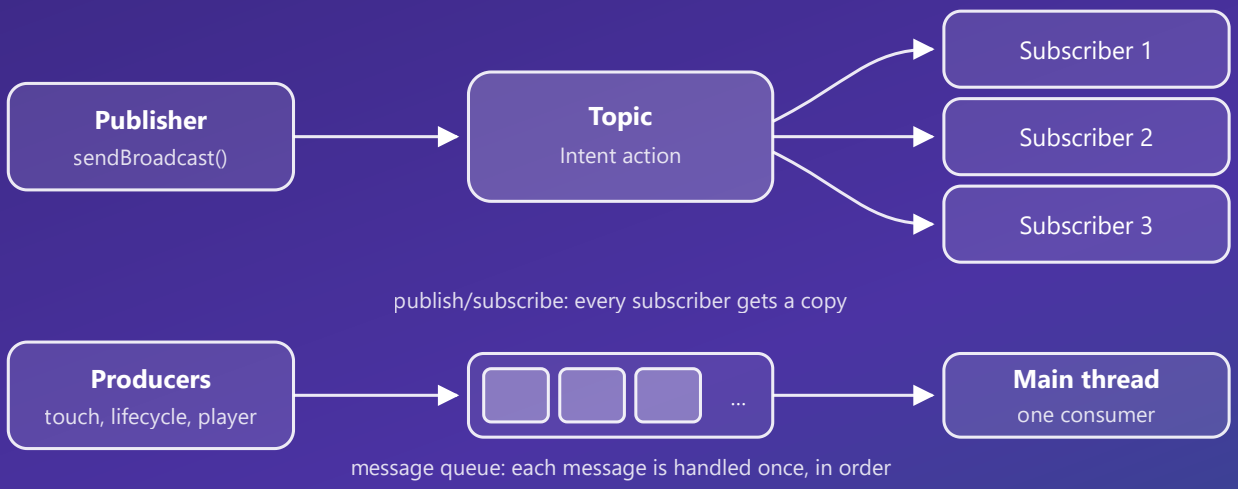
CSE 489 · MOBILE APPLICATION DEVELOPMENT · BRAC UNIVERSITY

Assignment 2, explained from the inside

Every feature of the app (navigation drawer, broadcast receivers, pinch-to-zoom image, video and audio players), the exact code that makes each one work, and how all of it connects to the theory of **Publish/Subscribe** and **Message Queueing**.



Project: LabAssignment2_CSE489 (Kotlin · XML Views · Material 3 · Android Jetpack)
Covers: all 21 Kotlin source files of the app, the layouts and resources, the build setup and the 44 automated tests
Written: 28 October 2026

Contents

How to read this document 4

Part 1 · The project at a glance 5

1.1 What the assignment asks for 5
1.2 What the finished app does 5
1.3 Tools and libraries 6
1.4 How the pieces fit together 7
1.5 Where everything lives 7

Part 2 · Theory: message queues and publish/subscribe 9

2.1 Why programs send messages 9
2.2 Message queues (point-to-point) 9
2.3 Publish/subscribe 10
2.4 Three kinds of decoupling 11
2.5 Delivery promises, retained and lost messages 12
2.6 The Observer pattern and the event loop 13

Part 3 · Android is built on messages 15

3.1 An Intent is a message 15
3.2 The broadcast system is a pub/sub broker 16
3.3 The main thread is a message queue 17
3.4 Lifecycles decide when you subscribe 18
3.5 The big mapping table 19

Part 4 · The app skeleton: drawer, navigation and state 21

4.1 MainActivity, the single host 21
4.2 Navigation graph and drawer menu 22
4.3 Fragments and ViewBinding 24
4.4 Back closes the drawer first 25
4.5 ViewModel, SavedStateHandle, LiveData and StateFlow 26
4.6 Edge-to-edge and window insets 28

Part 5 · Feature A: Broadcast Receiver 30

5.1 The whole flow in one picture 30
5.2 Screen 1: the Spinner and Proceed 30
5.3 Activity 2 (custom): typing the message 32
5.4 CustomMessageReceiver: the subscriber 34
5.5 Activity 3: subscribe, publish, receive 35
5.6 The lost-message problem 38
5.7 Rotation: no duplicates, nothing lost 39
5.8 Battery: a system topic with a retained message 40
5.9 BatteryStatusLiveData: subscribing only while visible 41
5.10 Turning the battery Intent into a percentage 43

5.11 Who may publish and who may listen 44

Part 6 · Feature B: Image Scale 46

6.1 Downloading the picture with Coil 46
6.2 The image Matrix and the fit 48
6.3 Pinch zoom with ScaleGestureDetector 49
6.4 Pan, edges and double-tap 51
6.5 Touch events are a message stream 53
6.6 Keeping the zoom through rotation 54

Part 7 · Feature C: Video 57

7.1 VideoView and MediaController 57
7.2 Pausing, remembering, resuming 58
7.3 Player events, errors and the drawer 60

Part 8 · Feature D: Audio 62

8.1 MediaPlayer is a state machine 62
8.2 The player lives in the ViewModel 63
8.3 State flows up, commands flow down 64
8.4 The screen: buttons, slider and times 67
8.5 Audio and video never play together 68

Part 9 · Helpers, resources and build setup 69

9.1 Utility functions 69
9.2 Resources: strings, layouts, themes 69
9.3 The manifest 71
9.4 Gradle build and dependencies 72

Part 10 · Tests 74

10.1 Unit tests 74
10.2 Instrumented tests 75
10.3 Why tests must wait for messages 76

Part 11 · Summary and revision 77

11.1 One-page cheat sheet 77
11.2 Likely viva questions 78
11.3 Glossary 80
11.4 How to run and try every feature 81

Tip: click any line above to jump to it. The PDF also has bookmarks for every part and section.

How to read this document

This document explains the whole project so that you could rebuild it, defend it in a viva, and connect it to the theory of messaging systems. You do not need to know anything about Pub/Sub or message queues before you start: Part 2 teaches them from zero.

The plan

1. **Part 1** shows what the assignment asked for and what the finished app does.
2. **Part 2** teaches the theory: message queues, publish/subscribe, decoupling, delivery guarantees, the Observer pattern and the event loop.
3. **Part 3** shows that Android itself is built from these ideas. After this part, every feature of the app will look like a messaging problem.
4. **Parts 4 to 8** go through the app feature by feature. Each feature follows the same recipe: *what you see*, *how it works step by step*, *the code responsible* and *the Pub/Sub and message-queue view*.
5. **Parts 9 and 10** cover the helpers, resources, build files and the 44 tests.
6. **Part 11** is for revision: a one-page cheat sheet, viva questions with answers, a glossary and a checklist for trying every feature.

The coloured boxes

IN SIMPLE WORDS

A plain-language explanation of an idea, usually with an everyday comparison.

PUB/SUB & MESSAGE-QUEUE VIEW

How the code you just read is an example of the messaging theory from Part 2.

CODE RESPONSIBLE

Which file, function and lines make the feature work.

REMEMBER

The key points to keep in your head, often the answers to viva questions.

COMMON MISTAKE

What goes wrong if you write the code the obvious way, and why the project avoids it.

About the code listings

Every code listing is copied **directly from the project files** by line number, so the line numbers in the grey header match the real file. Lines with a yellow background are the most important ones in that listing. A `⋮` line means some lines in between were left out. Unless a path starts with `res/`, `test/` or `androidTest/`, it is relative to `app/src/main/java/com/example/labassignment2_cse489/`.

PART 1

The project at a glance

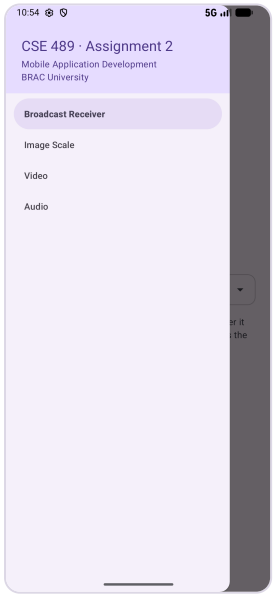
1.1 What the assignment asks for

The brief (*CSE 489 Assignment L2 v5*) asks you to practise **fragments**, the **navigation drawer** and the **Android Jetpack** way of writing apps. The drawer must have four items:

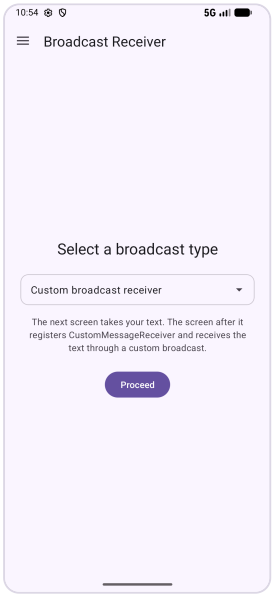
Drawer item	What the brief requires
A. Broadcast Receiver	A Spinner with two choices, *Custom broadcast receiver* and *System battery notification receiver*, and a button to go on. **Activity 2:** for Custom, take plain text from the user and pass it on; for Battery, receive the battery percentage broadcast. **Activity 3:** for Custom, *create the custom broadcast receiver* that receives the text from Activity 2; for Battery, do nothing.
B. Image Scale	Load an image from the internet and scale it with a pinch gesture.
C. Video	Play one video inside the app (media player).
D. Audio	Play one audio track inside the app (media player).

Feature A is the heart of this document, because a *broadcast* is Android's built-in publish/subscribe system. But you will see that the other three features are full of messaging ideas too.

1.2 What the finished app does



The navigation drawer



Broadcast Receiver (start screen)

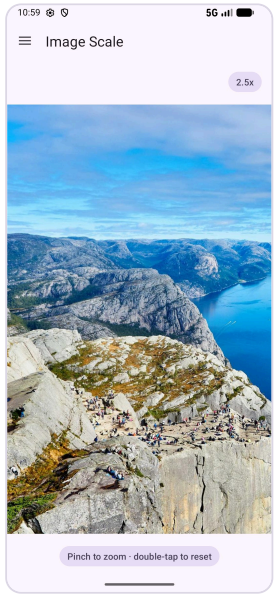
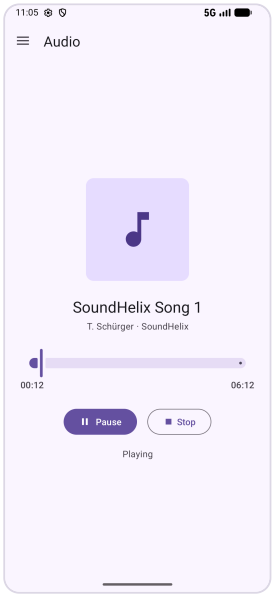


Image Scale, zoomed to 2.5x



Audio player

- **One main activity** (`MainActivity`) holds a toolbar, a navigation drawer and a container. Choosing a drawer item swaps a **fragment** into the container.
- **Broadcast Receiver**: pick a type, tap *Proceed*. Custom: type a message → the next activity registers a `BroadcastReceiver` and receives your message through a real broadcast. Battery: see the live battery percentage, charging status and the time of the last update.

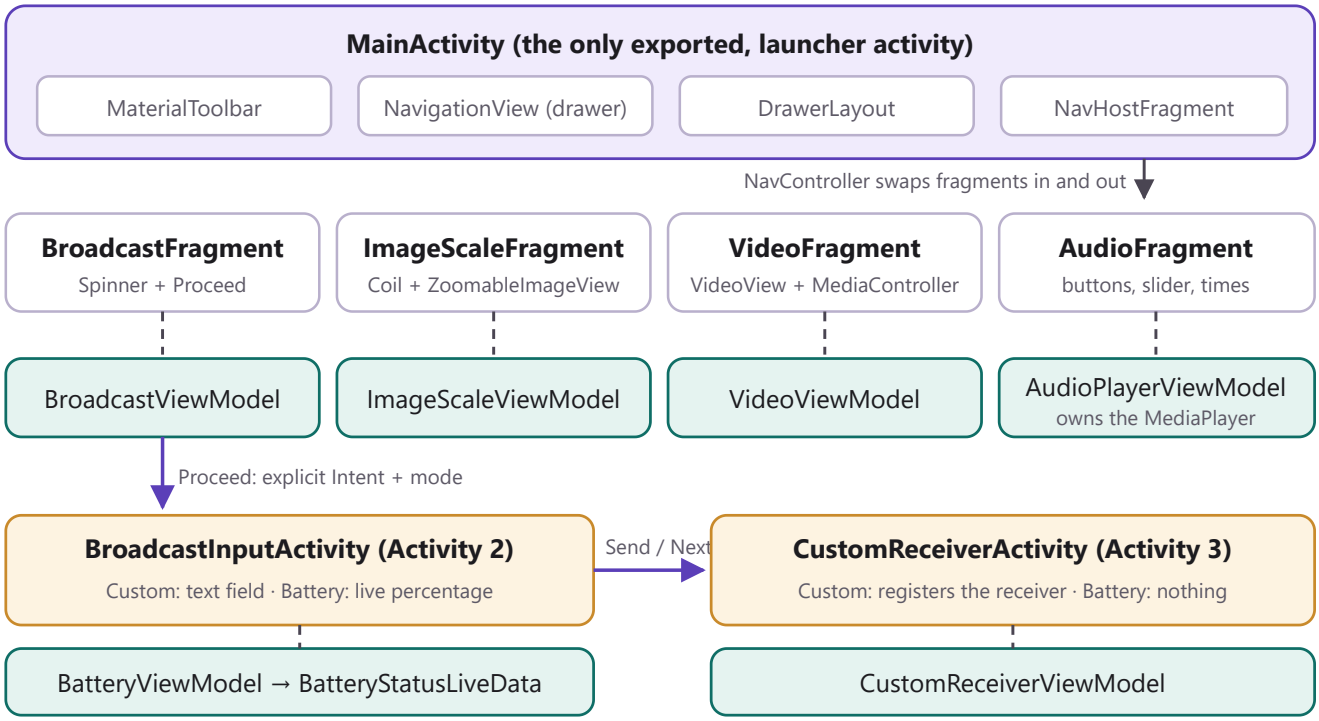
- **Image Scale**: downloads a 2000 × 1300 photo, then lets you pinch to zoom (1x to 5x), drag to pan and double-tap to zoom in or reset. Shows a loading spinner, and an error with *Retry* when offline.
- **Video**: plays a bundled trailer with the standard Android control bar. It pauses when you leave and continues from the same moment.
- **Audio**: plays a bundled MP3 with Play/Pause, Stop, a seek slider and mm:ss times. Music keeps playing through a screen rotation but pauses when you leave the screen.
- Works in **portrait and landscape**, in **light and dark** themes, and draws **edge-to-edge** behind the system bars.

1.3 Tools and libraries

Thing	What it is, in one line
Kotlin	The programming language of every source file.
XML Views + ViewBinding	Screens are drawn from XML layout files; ViewBinding generates a typed class per layout, so no `findViewById`.
Material 3 (Material Components)	Google's design system: toolbar, buttons, cards, slider, progress ring, DayNight theme.
AndroidX Navigation	A graph of screens plus `NavigationUI`, which connects it to the drawer and toolbar.
AndroidX Fragment, Activity	Fragments for the four screens; `enableEdgeToEdge()`, `OnBackPressedDispatcher`, `by viewModels()`.
AndroidX Lifecycle	`ViewModel`, `SavedStateHandle`, `LiveData`, `lifecycleScope`, `repeatOnLifecycle`.
Kotlin coroutines + Flow	Background work without blocking (`launch`, `delay`) and `StateFlow` for screen state.
Coil 3 (+ OkHttp)	Downloads and decodes the internet picture.
`android.media.MediaPlayer`	The platform audio/video player (used directly for audio, inside `VideoView` for video).
JUnit 4, Espresso	Unit tests on the computer; UI tests on an emulator or phone.
Gradle (AGP 9.4), SDK 37	Build system. `minSdk 24` (Android 7.0), `targetSdk` and `compileSdk 37`.

No Jetpack Compose is used. The brief links a Compose resource list, but asks for fragments and the navigation drawer, which belong to the View system.

1.4 How the pieces fit together



Purple: activities · White: fragments (drawer screens) · Teal: ViewModels (state that survives rotation) · Amber: the broadcast flow's extra activities

Figure 1.1 — One activity hosts the four drawer screens. Only the broadcast flow opens extra activities.

IN SIMPLE WORDS

Think of `MainActivity` as a picture frame with a menu on its side. The four screens are pictures you slide in and out of the frame. Each picture has a "memory card" (its ViewModel) that keeps its data safe while the picture is taken out and put back, for example when you rotate the phone. Only the Broadcast Receiver feature leaves the frame, opening two separate screens (activities), because the brief explicitly calls them "activities".

1.5 Where everything lives

app/src/main/java/com/example/labassignment2_cse489/ — 21 Kotlin files

```
MainActivity.kt                Drawer + toolbar + NavHostFragment, Back handling, insets
broadcast/
  BroadcastFragment.kt         Screen 1: Spinner + Proceed
  BroadcastViewModel.kt        Spinner selection (SavedStateHandle)
  BroadcastMode.kt             The two choices: CUSTOM / BATTERY
  BroadcastInputActivity.kt    Activity 2: text input, or live battery level
  BatteryStatus.kt             Percentage maths + charging-status mapping
  BatteryStatusLiveData.kt     Subscribes to the battery broadcast only while observed
  BatteryViewModel.kt          Keeps that LiveData across rotation
  CustomReceiverActivity.kt    Activity 3: registers the receiver, sends the broadcast
  CustomMessageReceiver.kt     The custom BroadcastReceiver (the subscriber)
  CustomReceiverViewModel.kt   "Already sent?" + last received message
image/
  ImageScaleFragment.kt        Loading / error / zoom label
  ImageScaleViewModel.kt       Coil download exposed as a StateFlow
  ZoomableImageView.kt         ScaleGestureDetector + GestureDetector + Matrix
  ZoomMath.kt                  Pure zoom maths (unit-tested)
```

```
video/
  VideoFragment.kt              VideoView + MediaController
  VideoViewModel.kt             Saved playback position
audio/
  AudioFragment.kt              Buttons, slider, times
  AudioPlayerViewModel.kt       Owns the MediaPlayer, publishes AudioUiState
util/
  TimeFormat.kt                 mm:ss and clock-time formatting
  WindowInsetsExt.kt            Edge-to-edge padding helper
```

Resources live in `app/src/main/res/`: `layout/` (screens), `layout-land/` (landscape versions of the battery and audio screens), `navigation/nav_graph.xml`, `menu/drawer_menu.xml`, `values/` (strings, the spinner's string-array, colours, sizes, theme) and `raw/` (the bundled `sample_video.mp4` and `sample_audio.mp3`). Unit tests are in `app/src/test/`, instrumented (on-device) tests in `app/src/androidTest/`.

PART 2

Theory: message queues and publish/subscribe

Before looking at Android, let's learn the two classic ways programs talk to each other by sending messages. Everything here is general computer science; Part 3 then shows where Android uses each idea.

2.1 Why programs send messages

The simplest way for one piece of code (A) to ask another piece (B) to do something is a **direct call**: `b.doWork()`. That is fine inside one small program, but it ties A and B tightly together:

- A must **know** B (have a reference to it, know its name and address).
- B must **exist and be running** at the moment A calls.
- A usually **waits** until B has finished.

The alternative is to send a **message**: A writes down what happened or what it wants and hands it to a **channel**. Some other code picks it up from the channel and acts on it. The software that runs these channels is called a **message broker** or *message-oriented middleware*.

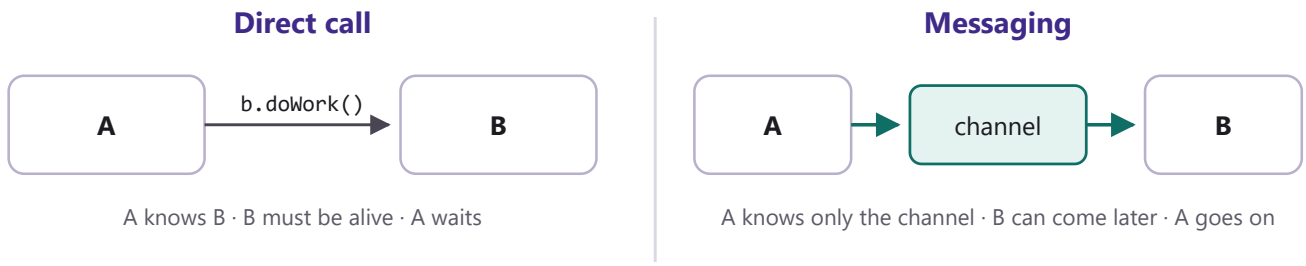


Figure 2.1 — A direct call couples A to B. A message goes through a channel that sits between them.

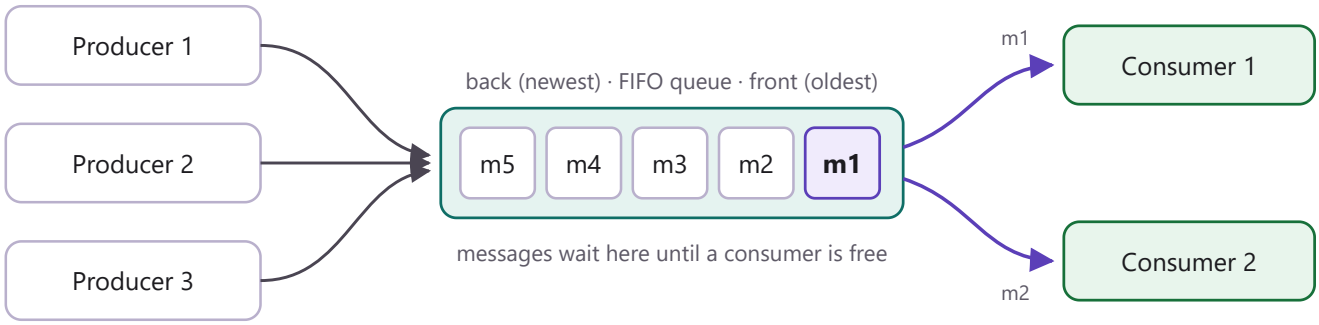
A message usually has two parts, like a letter in an envelope:

- a **header** (the envelope): where it should go, what kind of message it is, when it was sent;
- a **body** or **payload** (the letter): the actual data.

The code that sends messages is the **producer** (or *publisher*); the code that receives them is the **consumer** (or *subscriber*). There are two main styles of channel: the **queue** and the **topic**.

2.2 Message queues (point-to-point)

A **message queue** is a waiting line for messages. Producers add messages at the back; consumers take them from the front, usually in **FIFO** order (first in, first out). The key rule is: **each message is handled by exactly one consumer**. That is why this style is called **point-to-point**.



Competing consumers: m1 goes to Consumer 1 only, m2 to Consumer 2 only. Nobody gets a copy.

Figure 2.2 — A message queue: many producers, a waiting line, and each message delivered to one consumer.

IN SIMPLE WORDS

A message queue is the token line at a bank. Customers (messages) join the back of the line. Whenever a counter (consumer) is free, it calls the person at the front. Each customer is served by *one* counter, never by two. If many customers arrive at once, they simply wait in the line instead of being turned away.

What a queue gives you

Property	Meaning
Buffering	The producer can be faster than the consumer for a while; extra messages wait in the queue (also called *load levelling*).
Ordering	Messages are taken in the order they arrived (FIFO), unless the queue is sorted by priority or time.
One consumer per message	Work is never done twice. Several consumers on one queue share the work (*competing consumers*).
Acknowledgement	In real brokers the consumer confirms ("ack") that it finished; if it crashes first, the message is given to someone else.
Durability	A broker can save messages to disk, so they survive a restart.

Real examples: RabbitMQ queues, Amazon SQS, Java JMS queues, and, inside every operating system with a graphical interface, the **UI message queue** of the main thread (Windows' message loop, the browser's event loop, Android's `Looper`). That last one is extremely important for this app (section 3.3).

2.3 Publish/subscribe

In **publish/subscribe** (pub/sub), the channel is a **topic**: a named subject such as "battery changed" or "sports news". Consumers **subscribe** to the topics they care about. A producer **publishes** a message to a topic without knowing who, or how many, will read it. The broker then delivers **a copy to every subscriber** of that topic. This one-to-many copying is called **fan-out**.

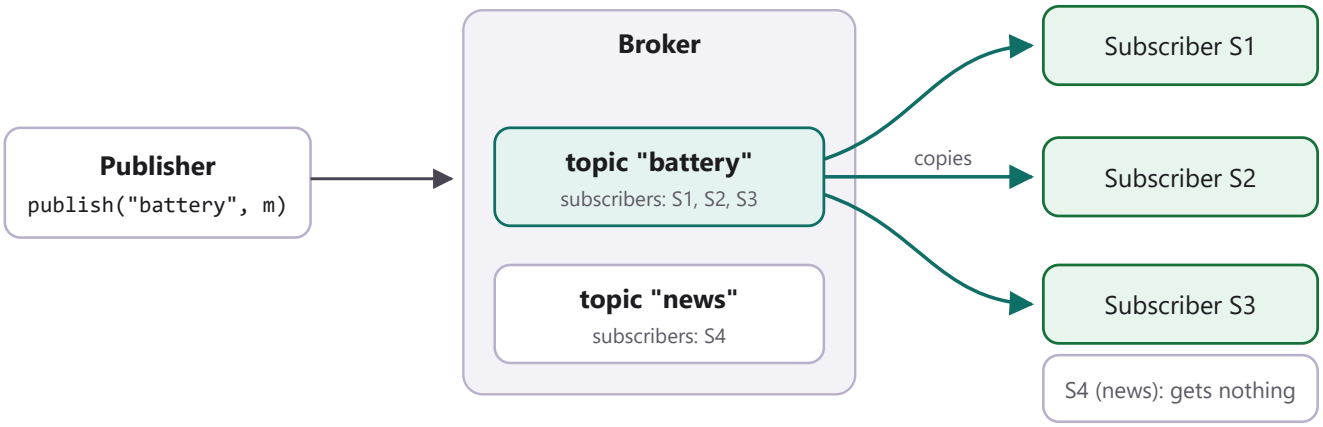


Figure 2.3 — Publish/subscribe: the publisher only names a topic; the broker copies the message to every subscriber of that topic.

IN SIMPLE WORDS

Pub/sub is a radio station. The station (publisher) broadcasts on a frequency (topic). Every radio tuned to that frequency (subscriber) hears the same programme at the same time. The station has no idea how many radios are listening. And if your radio was switched off, you missed that programme: the station does not record it for you. Keep this last point in mind; it explains a key design decision in Feature A.

How subscribers choose messages

- **Topic-based**: you subscribe to a name, and receive everything published under it.
- **Content-based**: you subscribe with a condition on the message itself (for example, "only messages where `type = image/*`").

Real examples: Apache Kafka topics, Google Cloud Pub/Sub, MQTT (used by IoT devices), Redis Pub/Sub, Firebase Cloud Messaging topics, and **Android broadcasts**.

Queue or topic?

	Message queue (point-to-point)	Publish/subscribe (topic)
Who receives a message?	Exactly **one** consumer	**Every** current subscriber gets a copy
Typical use	Jobs to do: "resize this photo", "send this email"	Events to announce: "battery changed", "user logged in"
Sender's view	"Someone please do this"	"This happened; whoever cares, react"
Adding a receiver	Shares the work (each gets fewer messages)	Adds a listener (each still gets all messages)
Everyday picture	Bank token line	Radio station, YouTube channel, WhatsApp group

2.4 Three kinds of decoupling

The main reason people use messaging is **decoupling**: making the sender and the receiver depend on each other as little as possible. A famous survey paper, "*The Many Faces of Publish/Subscribe*" (Eugster, Felber, Guerraoui and Kermarrec, ACM Computing Surveys, 2003), names three separate kinds:

Decoupling in...	Meaning	Android example in this app
Space	Publisher and subscriber do not know each other. They share only the topic name.	`sendBroadcast()` holds no reference to `CustomMessageReceiver`; both just use the action string.
Time	They do not have to be running at the same moment; the broker keeps messages for later.	**Missing** for normal broadcasts (a receiver that registers late gets nothing). **Partly** present for the sticky battery broadcast, which keeps its last copy.
Synchronization	The publisher is not blocked while the message is delivered; subscribers are told asynchronously, later.	`sendBroadcast()` returns at once; `onReceive()` runs a moment later on the main thread.

REMEMBER

Android broadcasts give you **space** and **synchronization** decoupling, but **not time** decoupling. This single fact explains why, in this app, Activity 3 sends the custom broadcast itself instead of Activity 2 (section 5.6).

2.5 Delivery promises, retained and lost messages

How many times is a message delivered?

Guarantee	Meaning	Cost
At-most-once	Sent once, never retried. It may be *lost*, but never duplicated ("fire and forget").	Cheapest. Android broadcasts behave like this: if no receiver is registered, the message is simply dropped.
At-least-once	Retried until the consumer confirms. Never lost, but may arrive *twice*.	Consumers must be **idempotent** (handling a message twice does no harm) or remove duplicates.
Exactly-once	Delivered once and only once.	Hardest; needs duplicate detection plus transactions.

Duplicates do not only come from the broker. If your own code can run twice (for example because Android recreates a screen when you rotate the phone), it can *publish* the same message twice. Section 5.7 shows how the app prevents exactly that with a saved "already sent" flag, which is a simple form of **de-duplication**.

Durable, non-durable and retained

- A **durable subscription**: the broker stores messages for a subscriber while it is offline and hands them over when it comes back (like a voicemail box).
- A **non-durable subscription**: messages are delivered only to subscribers that are connected *right now*; everyone else misses them (like live radio).
- A **retained** (or *last-value*) message: the broker keeps **only the latest** message of a topic and gives it to every **new** subscriber immediately. MQTT calls this the *retain* flag. Android calls it a **sticky broadcast**.

- **Conflation**: when only the current state matters (a battery level, a playback position), older updates that were not yet delivered can be replaced by the newest one.

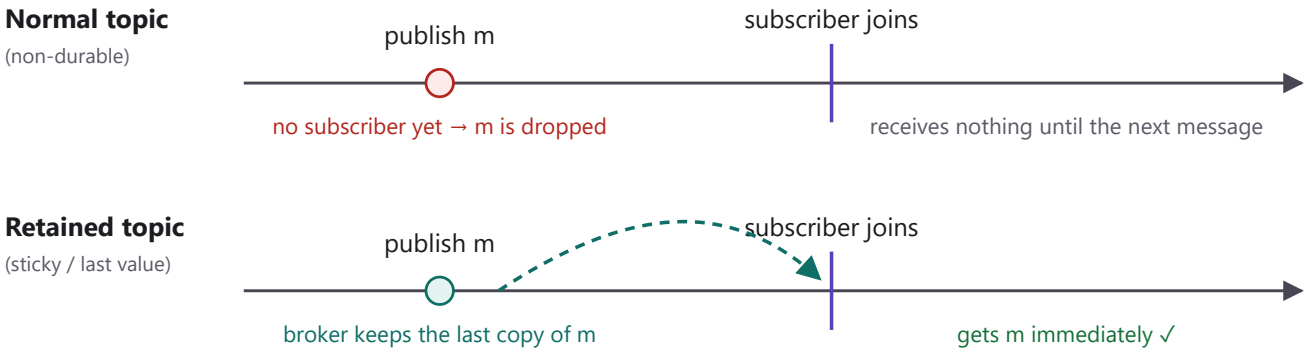


Figure 2.4 — A late subscriber misses a normal message but immediately receives a retained one.

WHERE THIS SHOWS UP IN THE APP

- The **custom broadcast** is a normal, non-durable message: it must be sent *after* the receiver has subscribed (section 5.6).
- The **battery broadcast** is retained (sticky): the moment the app subscribes, it gets the current battery level without waiting (section 5.8).
- **LiveData** and **StateFlow**, which hold every screen's state, also behave like retained, conflated topics: a new observer immediately gets the latest value, never a history.

2.6 The Observer pattern and the event loop

Observer: pub/sub without a broker

The **Observer** design pattern (from the "Gang of Four" book, 1994) is the small, in-memory cousin of pub/sub. An object called the **subject** keeps a list of **observers**. When its data changes, it calls a method on each observer. There is no separate broker: the subject *is* the topic, and everything happens inside one program.

	Observer pattern	Publish/subscribe
Middleman	None: the subject calls observers directly	A broker routes messages
Do they know each other?	The subject holds references to its observers	No: both know only the topic
Where	Inside one process	Often across processes or machines
In this app	`LiveData`, `StateFlow`, click listeners, drawer listeners, `onZoomChanged`	Broadcasts through the Android system

A **callback** or **listener** (`setOnClickListener`, `setOnPreparedListener`) is an observer with room for exactly one subscriber.

The event loop: one queue, one consumer

Graphical apps have one special thread, the **UI thread** (on Android: the **main thread**). Only this thread may touch the screen. Everything that happens (a touch, a timer, a network reply, a lifecycle change) is turned into a **message** and put into the UI thread's **message queue**. The thread runs a never-ending loop:

The idea of an event loop (pseudo-code)

```
while (true) {
    val message = queue.next()   // waits (sleeps) while the queue is empty
    handle(message)              // run the code for this message, then come back
}
```

This is a **message queue with exactly one consumer**. It has two big consequences:

1. **No locks needed.** Because only one thread handles UI messages, one at a time, two pieces of UI code can never run at the same moment and corrupt each other's data.
2. **Handlers must be quick.** While one message is being handled, every other message, including the user's touches, waits. If the main thread is blocked for about 5 seconds, Android shows an "App Not Responding" (ANR) dialog. Slow work (downloads, decoding) must run on another thread and send its result back as a new message.

Part 3 shows that Android's `Looper` and `MessageQueue` are exactly this loop, and that broadcasts, touches, player events, lifecycle callbacks and coroutine timers in this app all travel through it.

PART 3

Android is built on messages

Android apps are not one long program that runs from top to bottom. They are a set of components that the system talks to with messages. Once you see this, every part of the app becomes easy to explain.

3.1 An Intent is a message

An `Intent` is Android's message object. Its fields map neatly onto the parts of a message from section 2.1:

Intent field	Message idea	In this app
action (a string)	Topic / routing key: *what kind* of message	`"com.example.labassignment2_cse489.action.CUSTOM_MESSAGE"`, `Intent.ACTION_BATTERY_CHANGED`
extras (key → value)	Body / payload	The typed text (`EXTRA_MESSAGE`), the mode (`"mode"`), the battery level and scale
component (a class)	An exact address: one named recipient	`Intent(context, CustomReceiverActivity::class.java)`
package	Restricts who may receive it	`.setPackage(context.packageName)` : only this app

The same message object is used in two very different ways:

EXPLICIT INTENT = POINT-TO-POINT

`startActivity(intent)` with a component names exactly one receiver. The system delivers it to that one activity, like a message on a queue with one consumer. The app uses this to go from Screen 1 to Activity 2 to Activity 3.

BROADCAST INTENT = PUBLISH/SUBSCRIBE

`sendBroadcast(intent)` names only an action. The system delivers a copy to *every* receiver whose filter matches that action, like a topic. The app uses this for the custom message and receives the system's battery broadcast this way.

CODE RESPONSIBLE

The two kinds of Intent, side by side: an **explicit** Intent that opens Activity 3, and a **broadcast** Intent that is published to a topic.

`broadcast/CustomReceiverActivity.kt` lines 167–170

```
167 fun newIntent(context: Context, mode: BroadcastMode, message: String? = null): Intent =
168     Intent(context, CustomReceiverActivity::class.java)
169         .putExtra(EXTRA_MODE, mode.name)
170         .putExtra(EXTRA_MESSAGE, message)
```

broadcast/CustomMessageReceiver.kt lines 33–49

```kotlin
/** Our own action. The application ID prefix keeps it unique on the device. */
const val ACTION_CUSTOM_MESSAGE = "${BuildConfig.APPLICATION_ID}.action.CUSTOM_MESSAGE"

/** The extra that carries the user's text. */
const val EXTRA_MESSAGE = "${BuildConfig.APPLICATION_ID}.extra.MESSAGE"

/** The filter the receiver is registered with: it only matches our custom action. */
fun intentFilter(): IntentFilter = IntentFilter(ACTION_CUSTOM_MESSAGE)

/**
 * Builds the broadcast for [message]. setPackage() restricts delivery to this app, so no
 * other app can receive the user's text.
 */
fun createBroadcast(context: Context, message: String): Intent =
    Intent(ACTION_CUSTOM_MESSAGE)
        .setPackage(context.packageName)
        .putExtra(EXTRA_MESSAGE, message)
```

3.2 The broadcast system is a pub/sub broker

When you call `registerReceiver()` or `sendBroadcast()`, your app does not talk to the receiver directly. It sends a request over **Binder** (Android's way for processes to call each other) to the **ActivityManagerService**, which runs in the system's own process, `system_server`. That service is the broker:

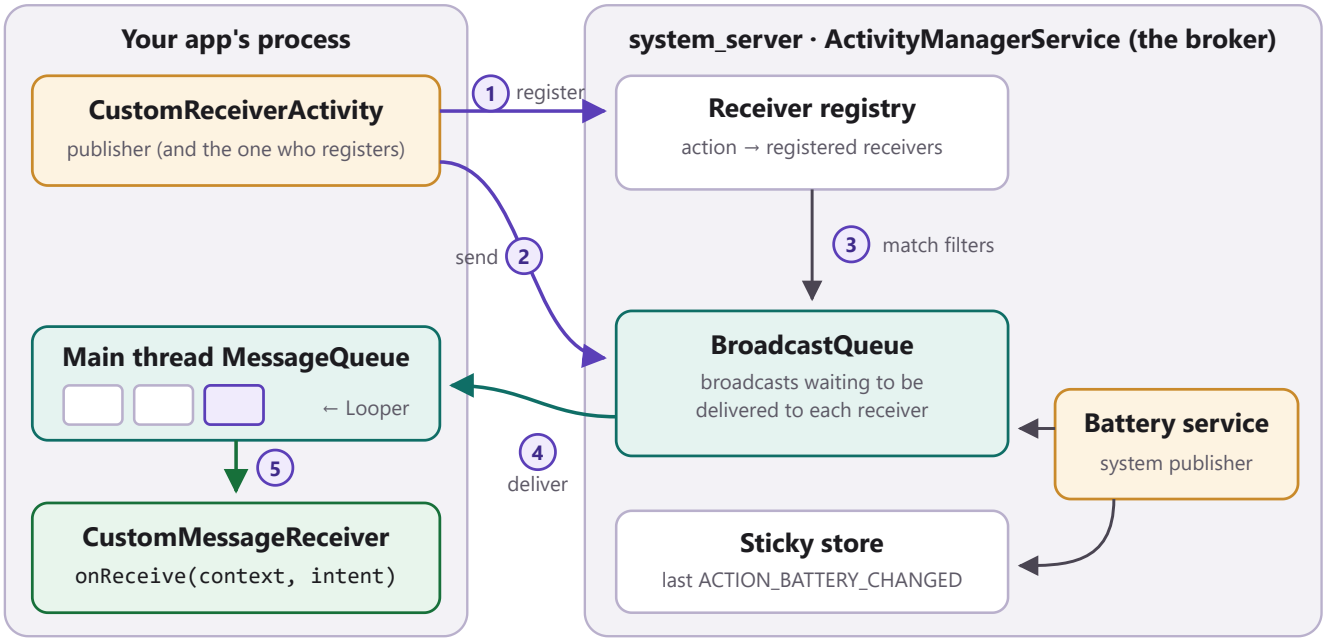


Figure 3.1 — The path of a broadcast. Steps 1–2 travel over Binder into the system; step 4 comes back over Binder; step 5 runs on your main thread.

1. **Subscribe.** `registerReceiver(receiver, IntentFilter(action))` stores "this receiver wants this action" in the system's registry.
2. **Publish.** `sendBroadcast(intent)` hands the Intent to the system and returns immediately. The sender does not wait for anyone to receive it.
3. **Route.** The system compares the Intent with every registered `IntentFilter` to find the matching receivers (for a sticky broadcast it also stores the Intent).

4. **Deliver.** The broadcast waits in a `BroadcastQueue` inside the system, then is handed to each receiver's process over Binder, where it is posted onto that process's main-thread message queue.
5. **Consume.** When the main thread's `Looper` reaches that message, it calls `onReceive()`.

Two kinds of subscription

	Manifest-declared receiver	Context-registered receiver (used in this app)
Declared with	`<receiver>` in `AndroidManifest.xml`	`registerReceiver(...)` in Kotlin code
Messaging view	A **static, permanent** subscription; the system can even start the app to deliver	A **dynamic** subscription that exists only between register and unregister
Limits	Since Android 8, most implicit broadcasts are no longer delivered this way; `ACTION_BATTERY_CHANGED` never is	Receives anything it registered for, while registered
Clean-up	None needed	**Must** unsubscribe (`unregisterReceiver`), or the receiver leaks

Two other flavours exist but are not used here: **ordered broadcasts** (`sendOrderedBroadcast`) are delivered to one receiver at a time in priority order, and any receiver may stop the chain, which is closer to a queue than to fan-out. **LocalBroadcastManager** was an in-process event bus that is now deprecated.

3.3 The main thread is a message queue

Section 2.6 described an event loop. On Android it is made of four classes:

- `Message`: one item of work (or a `Runnable` to run), with the time it should run.
- `MessageQueue`: the queue of messages, kept **sorted by time**, so a delayed message waits until its moment.
- `Handler`: the producer side: `post()` / `sendMessageDelayed()` put messages in.
- `Looper`: the consumer side: the loop that takes the next message and runs it, forever.

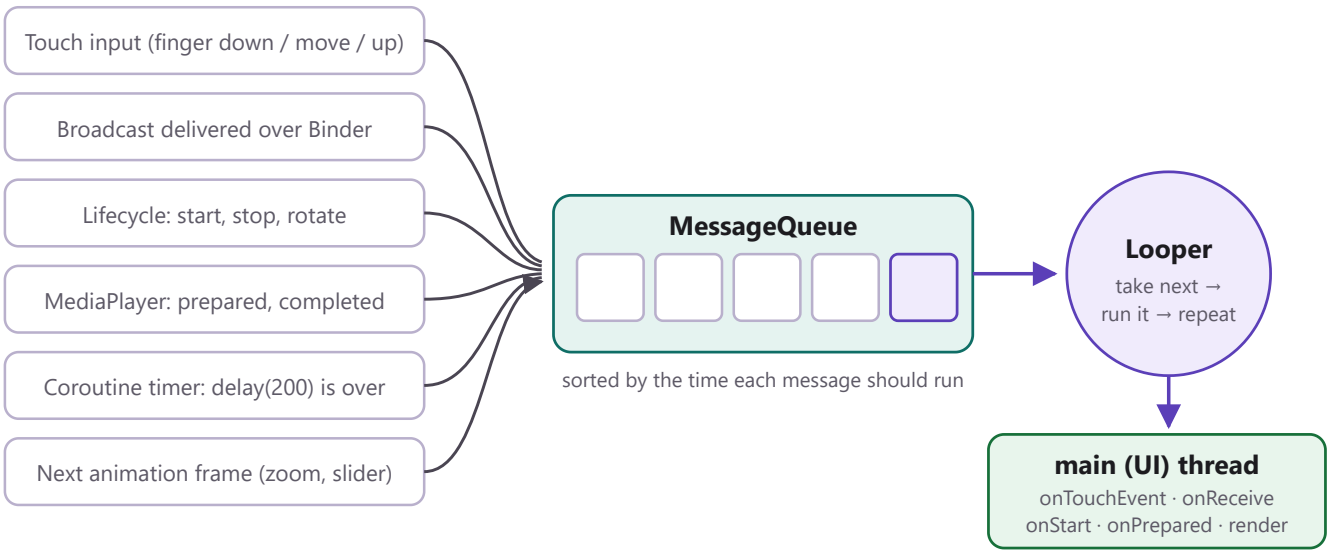


Figure 3.2 — One queue, one consumer. Every callback in this app runs because the main Looper took a message from this queue.

The project relies on this model in many places, and its comments say so:

broadcast/BatteryStatusLiveData.kt lines 22–27

```
22  private val receiver = object : BroadcastReceiver() {
23      override fun onReceive(context: Context, intent: Intent) {
24          // Context-registered receivers run on the main thread, so setValue() is safe here.
25          value = BatteryStatus.fromIntent(intent)
26      }
27  }
```

audio/AudioPlayerViewModel.kt lines 64–69

```
64  player.setOnPreparedListener { onPrepared() }
65  player.setOnCompletionListener { onCompleted() }
66  player.setOnErrorListener { _, what, extra -> onError(what, extra) }
67  // Initialized -> Preparing. prepareAsync() returns at once, so the UI never waits; the
68  // player calls onPrepared (on the main thread) when it is ready to play.
69  player.prepareAsync()
```

- A context-registered receiver's `onReceive()` runs on the main thread, so it may update a `LiveData` with `value = …` (which is only allowed on the main thread).
- `prepareAsync()` returns at once; later the player posts an "I am prepared" message, and `onPrepared()` runs on the main thread. This is **asynchronous request/reply**: send a request, get the answer as a message.
- The audio progress loop calls `delay(200)`. On the main dispatcher this is a delayed message in the same queue, so the loop never blocks the screen.
- Touch events for the zoomable image, lifecycle callbacks such as `onStart`, and the frames of the double-tap zoom animation also arrive one by one through this loop.

REMEMBER

Android gives you **both** messaging styles. **Between components and apps**, broadcasts are **publish/subscribe**. **Inside your process**, all UI work is a **message queue with one consumer**: the main thread's Looper. A broadcast travels through both: the system's pub/sub routing first, then your main-thread queue.

3.4 Lifecycles decide when you subscribe

Activities and fragments are created, shown, hidden and destroyed by the system, and it tells you each step through a **lifecycle callback**. Callbacks come in matching pairs. The golden rule of this project is: **subscribe in one callback and unsubscribe in its partner**, so a subscription can never outlive the screen that needs it.

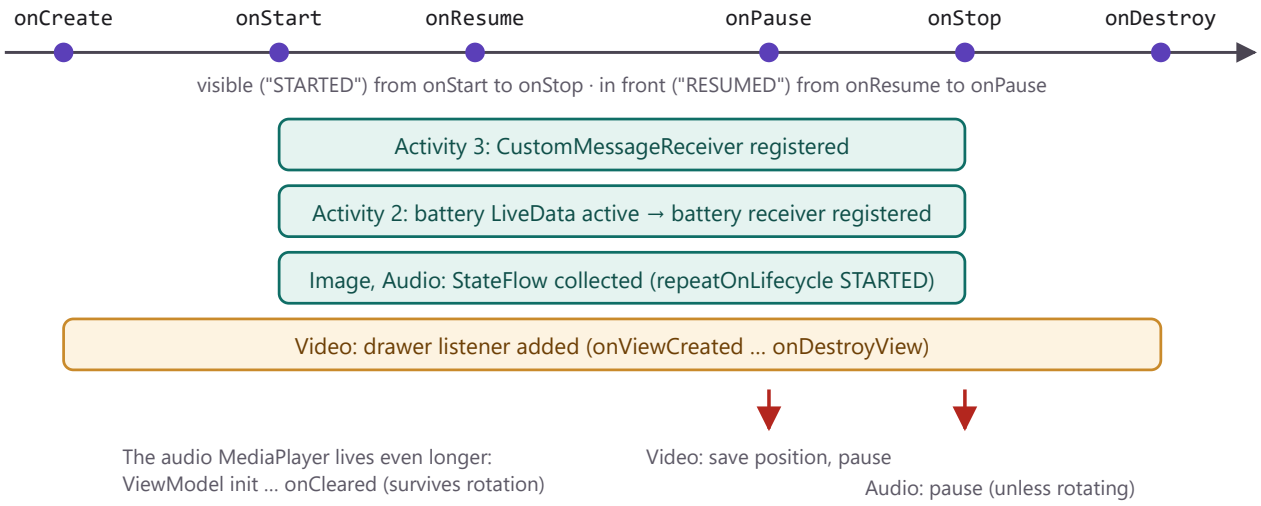


Figure 3.3 — Each subscription is opened and closed by a matching pair of callbacks.

COMMON MISTAKE

Registering a receiver in `onCreate` and forgetting to unregister it. The system keeps a reference to the receiver, which keeps the whole activity in memory after it is closed (a **leak**), and its `onReceive` keeps running for a screen nobody can see. In messaging terms: an abandoned subscription that still receives messages.

3.5 The big mapping table

This table is the bridge between theory and code. Every row is explained in detail later.

Messaging concept	Android equivalent	Where in this app
Message	`Intent` (action + extras)	`CustomMessageReceiver.createBroadcast()`
Topic	The Intent **action** string	`ACTION_CUSTOM_MESSAGE`, `ACTION_BATTERY_CHANGED`
Publisher	Whoever calls `sendBroadcast()`	`CustomReceiverActivity.sendCustomBroadcast()`; the system for battery
Subscriber	`BroadcastReceiver`	`CustomMessageReceiver`; the receiver inside `BatteryStatusLiveData`
Subscribe / unsubscribe	`registerReceiver` / `unregisterReceiver`	Activity 3 `onStart` / `onStop`; LiveData `onActive` / `onInactive`
Subscription filter	`IntentFilter`	`CustomMessageReceiver.intentFilter()`
Broker	ActivityManagerService in `system_server`	Provided by Android
Non-durable delivery (late subscriber misses it)	Normal broadcast	Why Activity 3, not Activity 2, sends (5.6)
Retained / last-value message	Sticky broadcast	`ACTION_BATTERY_CHANGED` (5.8)
Access control on a topic	`setPackage`, `RECEIVER_NOT_EXPORTED`, protected broadcasts	5.11

Messaging concept	Android equivalent	Where in this app
De-duplication	A saved "already sent" flag	`CustomReceiverViewModel.initialBroadcastSent` (5.7)
Point-to-point message	Explicit Intent to start an activity	The `newIntent()` factories
Queue with one consumer	Main `Looper` + `MessageQueue`	Every callback in the app
Asynchronous request/reply	`prepareAsync()` → `onPrepared`; Coil `execute()`	Audio, Video, Image Scale
In-process topic with the latest value	`LiveData`, `StateFlow`	Every ViewModel
Subscription tied to a lifecycle	`observe(owner)`, `repeatOnLifecycle`	Activity 2, Activity 3, all fragments
Periodic producer	Coroutine loop with `delay()`	`AudioPlayerViewModel.startProgressUpdates()`
Fan-out to several observers	Destination listeners, drawer listeners	Toolbar + drawer follow the NavController (4.2)
One consumer wins	`OnBackPressedDispatcher`, touch "consumed" (`return true`)	Back closes the drawer (4.4); zoom gestures (6.5)
Persistent state	`SavedStateHandle`	Spinner choice, received message, video position

PART 4

The app skeleton: drawer, navigation and state

Before the four features, here is the frame that holds them: one activity, a drawer, the Navigation component, fragments, ViewModels and edge-to-edge drawing.

4.1 MainActivity, the single host

WHAT YOU SEE

A toolbar with a ☰ (hamburger) icon and a title. Tapping ☰, or swiping from the left edge, slides the drawer in with the four items. The screen below the toolbar changes when you pick an item.

HOW IT WORKS

The layout is a `DrawerLayout` with two children. The **first** child is the normal content (toolbar + a container). The child with `layout_gravity="start"` is the **drawer**, which slides in from the left. The container is a `FragmentContainerView` running a `NavHostFragment`: the Navigation component puts the current screen's fragment in it.

res/layout/activity_main.xml — lines 7–10, 15–18, 25–28, 31–39, 42–49

```
 7  <androidx.drawerlayout.widget.DrawerLayout xmlns:android="http://schemas.android.com/apk/res/android"
 8      xmlns:app="http://schemas.android.com/apk/res-auto"
 9      xmlns:tools="http://schemas.android.com/tools"
10      android:id="@+id/drawer_layout"
    ⋮
15      <LinearLayout
16          android:layout_width="match_parent"
17          android:layout_height="match_parent"
18          android:orientation="vertical">
    ⋮
25              <com.google.android.material.appbar.MaterialToolbar
26                  android:id="@+id/toolbar"
27                  android:layout_width="match_parent"
28                  android:layout_height="?attr/actionBarSize" />
    ⋮
31          <!-- Hosts the four screens; Navigation swaps fragments in and out of here. -->
32          <androidx.fragment.app.FragmentContainerView
33              android:id="@+id/nav_host_fragment"
34              android:name="androidx.navigation.fragment.NavHostFragment"
35              android:layout_width="match_parent"
36              android:layout_height="0dp"
37              android:layout_weight="1"
38              app:defaultNavHost="true"
39              app:navGraph="@navigation/nav_graph" />
    ⋮
42      <com.google.android.material.navigation.NavigationView
43          android:id="@+id/nav_view"
44          android:layout_width="wrap_content"
45          android:layout_height="match_parent"
46          android:layout_gravity="start"
47          app:headerLayout="@layout/nav_header"
48          app:menu="@menu/drawer_menu" />
```

```
</androidx.drawerlayout.widget.DrawerLayout>
```

`onCreate` does four things, in order:

MainActivity.kt lines 41–50

```kotlin
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)

    applyWindowInsets()
    setUpNavigation()
    setUpBackHandling()
}
```

1. `enableEdgeToEdge()`: draw behind the status and navigation bars (section 4.6).
2. `ActivityMainBinding.inflate(...)` + `setContentView`: build the views from `activity_main.xml` through ViewBinding (section 4.3).
3. `setUpNavigation()`: connect the drawer and toolbar to the Navigation component (4.2).
4. `setUpBackHandling()`: make Back close an open drawer first (4.4).

4.2 Navigation graph and drawer menu

The **navigation graph** is a list of all screens ("destinations"). It says which fragment class each one uses, what title (`android:label`) it shows, and which one opens first (`app:startDestination`).

res/navigation/nav_graph.xml lines 6–22, 30–35

```xml
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/nav_graph"
    app:startDestination="@id/broadcastFragment">

    <fragment
        android:id="@+id/broadcastFragment"
        android:name="com.example.labassignment2_cse489.broadcast.BroadcastFragment"
        android:label="@string/menu_broadcast"
        tools:layout="@layout/fragment_broadcast" />

    <fragment
        android:id="@+id/imageScaleFragment"
        android:name="com.example.labassignment2_cse489.image.ImageScaleFragment"
        android:label="@string/menu_image_scale"
        tools:layout="@layout/fragment_image_scale" />
⋮
    <fragment
        android:id="@+id/audioFragment"
        android:name="com.example.labassignment2_cse489.audio.AudioFragment"
        android:label="@string/menu_audio"
        tools:layout="@layout/fragment_audio" />
</navigation>
```

The **drawer menu** lists the four items. Look at the IDs: they are *exactly the same* as the destination IDs in the graph. That is the whole trick that connects them.

res/menu/drawer_menu.xml lines 6–21

```xml
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <group android:checkableBehavior="single">
        <item
            android:id="@+id/broadcastFragment"
            android:title="@string/menu_broadcast" />
        <item
            android:id="@+id/imageScaleFragment"
            android:title="@string/menu_image_scale" />
        <item
            android:id="@+id/videoFragment"
            android:title="@string/menu_video" />
        <item
            android:id="@+id/audioFragment"
            android:title="@string/menu_audio" />
    </group>
</menu>
```

Three lines of Kotlin wire everything together:

MainActivity.kt lines 59–71

```kotlin
private fun setUpNavigation() {
    // With FragmentContainerView the NavController must be read from the NavHostFragment.
    val navHostFragment =
        supportFragmentManager.findFragmentById(R.id.nav_host_fragment) as NavHostFragment
    val navController = navHostFragment.navController

    // Every drawer item is a top-level destination: all four show the hamburger icon (which
    // opens the drawer) instead of an Up arrow. Building the configuration from the menu
    // keeps this set in sync with drawer_menu.xml.
    val appBarConfiguration = AppBarConfiguration(binding.navView.menu, binding.drawerLayout)
    binding.toolbar.setupWithNavController(navController, appBarConfiguration)
    binding.navView.setupWithNavController(navController)
}
```

- `AppBarConfiguration(menu, drawerLayout)` declares every menu item a **top-level destination**. On a top-level screen the toolbar shows ☰ (which opens the drawer) instead of a back arrow. Building it from the menu keeps it in sync with `drawer_menu.xml` automatically.
- `toolbar.setupWithNavController(...)`: the toolbar title follows the current destination's `android:label`, and ☰ opens the drawer.
- `navView.setupWithNavController(...)`: tapping an item navigates to the destination with the same ID, highlights the current item and closes the drawer.

Each screen keeps its state. When you switch drawer items, NavigationUI saves the screen you leave (its fragment state and ViewModels) and restores it when you come back. That is why the Image Scale screen does not download the picture again, and why the Audio screen's player is still alive when you return. (It is also why the Audio screen must pause itself when you leave; see 8.5.)

PUB/SUB & MESSAGE-QUEUE VIEW

The `NavController` is a small **publisher**. Each time the destination changes it notifies every `OnDestinationChangedListener` it has. `setupWithNavController` subscribes **two** listeners: the toolbar (to change the title and icon) and the drawer (to move the highlight). One event, two subscribers: a **fan-out**. The NavController does not know what a toolbar is; it just announces "the destination is now Video". Tapping a drawer item goes the other way: it is a **command** sent to one receiver, the NavController ("navigate to `videoFragment`"), which is point-to-point.

4.3 Fragments and ViewBinding

A **fragment** is a reusable piece of screen with its own lifecycle, living inside an activity. The drawer swaps fragments inside one activity instead of starting a new activity for every screen, so the toolbar and the drawer never move.

A fragment has **two lifecycles**: the fragment object itself, and its **view**, which exists only from `onCreateView` / `onViewCreated` to `onDestroyView`. When you switch drawer items, the view of the screen you leave is destroyed but the fragment's saved state lives on. So every fragment in this project follows the same pattern:

broadcast/BroadcastFragment.kt lines 16–27, 69–72

```
16  class BroadcastFragment : Fragment(R.layout.fragment_broadcast) {
17
18      // ViewBinding for the fragment's view. It only exists between onViewCreated and
19      // onDestroyView, because a fragment can outlive its view (e.g. on the back stack).
20      private var _binding: FragmentBroadcastBinding? = null
21      private val binding get() = _binding!!
22
23      private val viewModel: BroadcastViewModel by viewModels()
24
25      override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
26          super.onViewCreated(view, savedInstanceState)
27          _binding = FragmentBroadcastBinding.bind(view)
    ⋮
69      override fun onDestroyView() {
70          super.onDestroyView()
71          _binding = null
72      }
```

- `Fragment(R.layout.fragment_broadcast)`: the constructor argument tells the fragment which layout to inflate, so no `onCreateView` is needed.
- **ViewBinding** generates a class per layout (`fragment_broadcast.xml` → `FragmentBroadcastBinding`) with one typed field per view ID, such as `binding.proceedButton`. No `findViewById`, no wrong casts.
- The binding is stored in a nullable `_binding` and set to `null` in `onDestroyView`, so the fragment never holds on to dead views (a memory leak).

PUB/SUB & MESSAGE-QUEUE VIEW

Fragments observe data with `observe(viewLifecycleOwner)`, not `observe(this)`. The subscription is tied to the **view's** lifecycle, so it ends when the view is destroyed. If it were tied to the fragment instead, every time the view was recreated a *new* subscription would be added while the old one stayed, and each update would be delivered several times: a classic duplicate-subscriber bug.

4.4 Back closes the drawer first

WHAT YOU SEE

With the drawer open, pressing Back closes the drawer and keeps you on the same screen. With the drawer closed, Back behaves normally (goes back to the start screen, or leaves the app).

HOW IT WORKS

On Android 16+ with `targetSdk` 36+, the old `onBackPressed()` method is never called (because of *predictive back*, the animation that previews where Back will take you). The supported way is an `OnBackPressedCallback` registered with the activity's `OnBackPressedDispatcher`. The trick is to keep it **enabled only while the drawer is open**:

MainActivity.kt lines 35–39, 73–84

```
35  private val closeDrawerOnBack = object : OnBackPressedCallback(false) {
36      override fun handleOnBackPressed() {
37          binding.drawerLayout.closeDrawer(GravityCompat.START)
38      }
39  }
    ⋮
73  private fun setUpBackHandling() {
74      onBackPressedDispatcher.addCallback(this, closeDrawerOnBack)
75      binding.drawerLayout.addDrawerListener(object : DrawerLayout.SimpleDrawerListener() {
76          override fun onDrawerOpened(drawerView: View) {
77              closeDrawerOnBack.isEnabled = true
78          }
79
80          override fun onDrawerClosed(drawerView: View) {
81              closeDrawerOnBack.isEnabled = false
82          }
83      })
84  }
```

After a rotation, `DrawerLayout` re-opens a drawer that was open but does not call the listener, so the callback is synchronised once more when the saved state is restored:

MainActivity.kt lines 52–57

```
52  override fun onRestoreInstanceState(savedInstanceState: Bundle) {
53      super.onRestoreInstanceState(savedInstanceState)
54      // After rotation DrawerLayout re-opens a drawer that was open, without notifying its
55      // listeners, so sync the Back callback with the restored state here.
56      closeDrawerOnBack.isEnabled = binding.drawerLayout.isDrawerOpen(GravityCompat.START)
57  }
```

PUB/SUB & MESSAGE-QUEUE VIEW

Back is the opposite of a broadcast. The `OnBackPressedDispatcher` keeps a stack of callbacks and gives each Back press to **only one** of them: the most recently added *enabled* callback. That is queue-like, "one consumer wins" delivery. Enabling and disabling the callback is like joining and leaving the line of consumers. The `DrawerListener` that does the toggling is itself a subscriber to the drawer's open/close events.

4.5 ViewModel, SavedStateHandle, LiveData and StateFlow

Why a ViewModel?

When you rotate the phone, Android **destroys** the activity and its fragments and creates new ones for the new screen shape. Anything stored in the activity or fragment is lost. A `ViewModel` survives this: the new screen receives the *same* ViewModel object. It is destroyed (`onCleared()`) only when the screen is gone for good.

broadcast/BroadcastFragment.kt — how every screen gets its ViewModel lines 23

```
23  private val viewModel: BroadcastViewModel by viewModels()
```

`by viewModels()` creates the ViewModel the first time and hands back the existing one after a rotation. An `AndroidViewModel` also gets the **Application** context (needed by Coil and MediaPlayer). A ViewModel must never keep an Activity or a View, because it outlives them.

SavedStateHandle: surviving process death

When the app is in the background, Android may kill its whole process to free memory. The ViewModel dies too. Values stored in its `SavedStateHandle` are written into a Bundle and given back when the user returns, so they survive even that.

broadcast/BroadcastViewModel.kt lines 13–30

```
13  class BroadcastViewModel(private val savedStateHandle: SavedStateHandle) : ViewModel() {
14
15      /** The broadcast type currently selected in the spinner. */
16      val selectedMode: LiveData<BroadcastMode> =
17          savedStateHandle.getLiveData(KEY_SELECTED_MODE, BroadcastMode.CUSTOM)
18
19      /** The current selection, read synchronously (e.g. when Proceed is tapped). */
20      val currentMode: BroadcastMode
21          get() = savedStateHandle[KEY_SELECTED_MODE] ?: BroadcastMode.CUSTOM
22
23      fun select(mode: BroadcastMode) {
24          savedStateHandle[KEY_SELECTED_MODE] = mode
25      }
26
27      private companion object {
28          const val KEY_SELECTED_MODE = "selected_mode"
29      }
30  }
```

LiveData and StateFlow: observable state

Both hold **the current value** of some state and notify observers when it changes.

	`LiveData`	`StateFlow`
Used in this app for	Spinner choice, battery status, received message	Image state, audio player state
Subscribe with	`liveData.observe(lifecycleOwner) { … }`	`flow.collect { … }` inside a coroutine
Lifecycle-aware?	Yes, by itself: only notifies observers that are STARTED or RESUMED	Not by itself: wrap with `repeatOnLifecycle(STARTED)`
Initial value	May be empty	Always has a value
Equal values	Delivered again	Skipped (only real changes are delivered)
Threading	`setValue` on the main thread only	Can be updated from any thread

The fragments collect their StateFlow like this. `repeatOnLifecycle(STARTED)` starts the block in `onStart` and **cancels** it in `onStop`, so the screen only listens while it is visible:

audio/AudioFragment.kt lines 38–43

```
38  // Collect the state only while the screen is visible (from onStart until onStop).
39  viewLifecycleOwner.lifecycleScope.launch {
40      viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
41          viewModel.uiState.collect { state -> render(state) }
42      }
43  }
```

PUB/SUB & MESSAGE-QUEUE VIEW

- A `LiveData` or `StateFlow` is an **in-process topic that retains its latest value**: the ViewModel publishes (`value = …` , `_uiState.update { … }`), the screen subscribes (`observe` , `collect`), and a new subscriber immediately receives the current value, exactly like the retained message of Figure 2.4.
- Both are **conflated**: a subscriber that is slow or temporarily stopped gets only the newest value, never a backlog of old ones. That is right for screen state, where only "now" matters.
- `observe(owner)` and `repeatOnLifecycle` make the **lifecycle the subscription manager**: subscribe at start, unsubscribe at stop, automatically.
- `SavedStateHandle` adds **durability**: the state survives a crash of the "subscriber side" (the process), like a broker persisting messages to disk.

What survives rotation, and where

Screen	What is kept	Kept in
Broadcast Receiver	Spinner choice	`BroadcastViewModel` (SavedStateHandle)
Activity 2 (battery)	Last battery value	`BatteryViewModel`
Activity 3 (custom)	Received text, count, "already sent"	`CustomReceiverViewModel` (SavedStateHandle)
Image Scale	Downloaded picture · zoom and position	`ImageScaleViewModel` · the view's own saved state
Video	Position, "keep playing?"	`VideoViewModel` (SavedStateHandle)

Screen	What is kept	Kept in
Audio	The MediaPlayer itself (music keeps playing)	`AudioPlayerViewModel`

4.6 Edge-to-edge and window insets

Since `targetSdk` 35, every app is drawn **edge-to-edge**: the status bar (top) and the navigation bar (bottom, or a side in landscape) are transparent and the app draws behind them. Each screen must therefore move its toolbar and controls out from under the bars. The amount of space the bars take is called the **window insets**. One helper does this for every screen:

util/WindowInsetsExt.kt lines 20–46

```kotlin
fun View.applyWindowInsetsPadding(
    left: Boolean = false,
    top: Boolean = false,
    right: Boolean = false,
    bottom: Boolean = false,
    includeIme: Boolean = false,
) {
    // Remember the XML padding once, so repeated inset passes don't keep adding to it.
    val initialLeft = paddingLeft
    val initialTop = paddingTop
    val initialRight = paddingRight
    val initialBottom = paddingBottom

    ViewCompat.setOnApplyWindowInsetsListener(this) { view, windowInsets ->
        var types = WindowInsetsCompat.Type.systemBars() or WindowInsetsCompat.Type.displayCutout()
        if (includeIme) types = types or WindowInsetsCompat.Type.ime()
        val insets = windowInsets.getInsets(types)
        view.updatePadding(
            left = initialLeft + if (left) insets.left else 0,
            top = initialTop + if (top) insets.top else 0,
            right = initialRight + if (right) insets.right else 0,
            bottom = initialBottom + if (bottom) insets.bottom else 0,
        )
        // Not consumed: sibling and child views still receive the same insets.
        windowInsets
    }
}
```

It remembers the padding from the XML once, then, every time new insets arrive, sets the padding to "XML padding + bar size" on the chosen sides. `includeIme` also adds the on-screen keyboard height (used by Activity 2, so the text field stays above the keyboard). `MainActivity` uses it like this:

MainActivity.kt lines 86–95

```kotlin
private fun applyWindowInsets() {
    // Deliver insets to every child the same way on old and new Android versions.
    ViewGroupCompat.installCompatInsetsDispatch(binding.root)
    // The app bar's background extends behind the status bar; its content sits below it.
    binding.appBar.applyWindowInsetsPadding(left = true, top = true, right = true)
    // Screens stay clear of the navigation bar (bottom in portrait, a side in landscape).
    binding.navHostFragment.applyWindowInsetsPadding(left = true, right = true, bottom = true)
    // Keep the drawer header text below the status bar while the drawer is open.
    binding.navView.getHeaderView(0).applyWindowInsetsPadding(top = true)
}
```

PUB/SUB & MESSAGE-QUEUE VIEW

Insets are an **event delivered down the view tree**: the window publishes "the bars now take this much space" and every view with a listener receives it. The listener returns `windowInsets` **unconsumed** (line 44), which means "I have seen it, pass the same message on", so siblings and children all get it too, like fan-out. A listener could instead *consume* the insets, which would stop the message there. `installCompatInsetsDispatch` makes old Android versions deliver insets to all siblings the same way new versions do.

PART 5

Feature A: Broadcast Receiver

This is the feature where the app uses Android's publish/subscribe system directly. It contains both kinds of topic from Part 2: a **custom, non-durable** topic owned by the app, and a **system, retained** topic owned by Android.

5.1 The whole flow in one picture

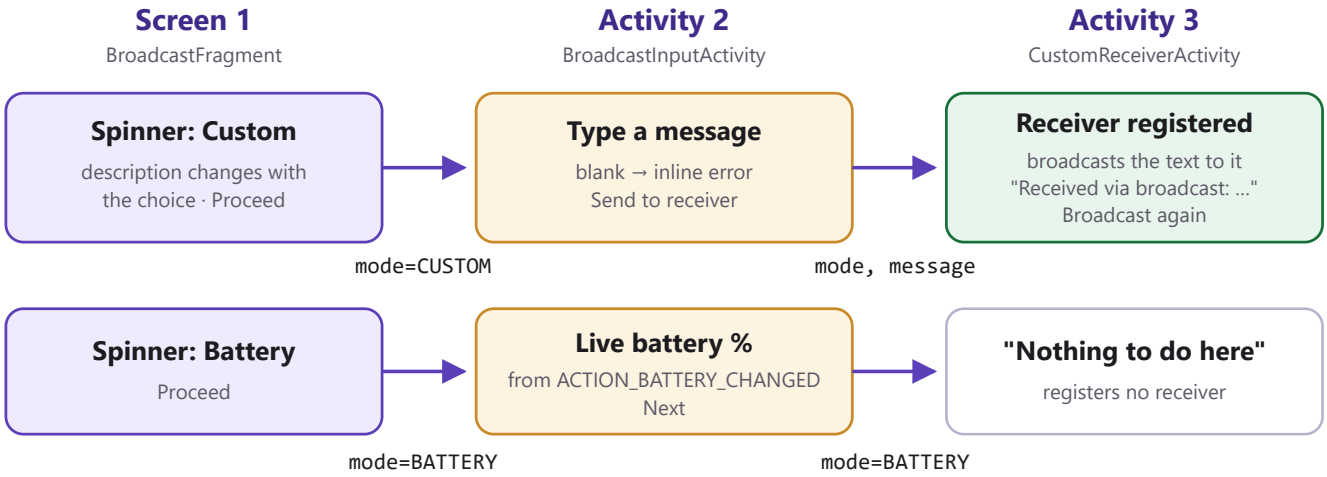
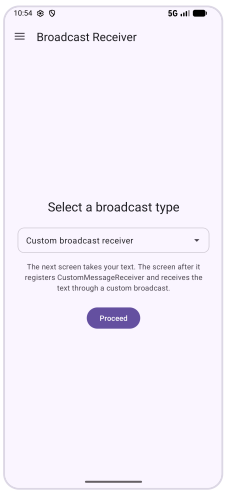


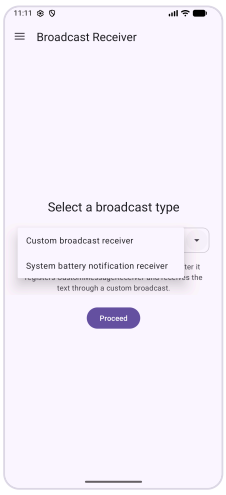
Figure 5.1 — Every arrow is an explicit Intent (point-to-point); the labels show the extras it carries.

The movement between screens is always **point-to-point** (explicit Intents). The **publish/subscribe** part happens in two places: inside Activity 3 (custom path) and inside Activity 2 (battery path).

5.2 Screen 1: the Spinner and Proceed



Custom selected



The two choices

HOW IT WORKS

The two Spinner items come from a **string-array** resource, in the same order as the `BroadcastMode` enum, so "position 0" always means CUSTOM and "position 1" BATTERY:

res/values/arrays.xml lines 3–7

```xml
<!-- Spinner items of the Broadcast Receiver screen, in the same order as BroadcastMode. -->
<string-array name="broadcast_types">
    <item>Custom broadcast receiver</item>
    <item>System battery notification receiver</item>
</string-array>
```

broadcast/BroadcastMode.kt lines 10–24

```kotlin
enum class BroadcastMode {
    /** The text typed in Activity 2 is broadcast to CustomMessageReceiver in Activity 3. */
    CUSTOM,

    /** Activity 2 shows the live battery level from the system's ACTION_BATTERY_CHANGED. */
    BATTERY;

    companion object {
        /** The mode at spinner [position], or null if the position is out of range. */
        fun fromPosition(position: Int): BroadcastMode? = entries.getOrNull(position)

        /** Parses a value written with [name]; null when it is missing or unknown. */
        fun fromName(name: String?): BroadcastMode? = entries.firstOrNull { it.name == name }
    }
}
```

The fragment fills the Spinner with an `ArrayAdapter`, shows the remembered choice, and reports every new choice to the ViewModel:

broadcast/BroadcastFragment.kt lines 42–62

```kotlin
private fun setUpSpinner() {
    // The ArrayAdapter takes its two items from the string-array in res/values/arrays.xml.
    val adapter = ArrayAdapter.createFromResource(
        requireContext(),
        R.array.broadcast_types,
        R.layout.item_spinner,
    )
    adapter.setDropDownViewResource(R.layout.item_spinner)

    val spinner = binding.broadcastTypeSpinner
    spinner.adapter = adapter
    // Show the remembered choice, e.g. after rotation.
    spinner.setSelection(viewModel.currentMode.ordinal, false)
    spinner.onItemSelectedListener = object : AdapterView.OnItemSelectedListener {
        override fun onItemSelected(parent: AdapterView<*>?, view: View?, position: Int, id: Long) {
            BroadcastMode.fromPosition(position)?.let(viewModel::select)
        }

        override fun onNothingSelected(parent: AdapterView<*>?) = Unit
    }
}
```

The ViewModel stores the choice in its `SavedStateHandle` (section 4.5). Its `selectedMode` LiveData then tells the fragment, which updates the description under the Spinner. Finally, *Proceed* opens Activity 2 with an explicit Intent carrying the mode:

broadcast/BroadcastFragment.kt lines 29–39, 64–67

```
29      setUpSpinner()
30      binding.proceedButton.setOnClickListener { openSelectedActivity() }
31
32      viewModel.selectedMode.observe(viewLifecycleOwner) { mode ->
33          binding.modeDescription.setText(
34              when (mode) {
35                  BroadcastMode.CUSTOM -> R.string.broadcast_custom_description
36                  BroadcastMode.BATTERY -> R.string.broadcast_battery_description
37              },
38          )
39      }
    ⋮
64  /** Opens Activity 2 with an explicit Intent that carries the selected mode. */
65  private fun openSelectedActivity() {
66      startActivity(BroadcastInputActivity.newIntent(requireContext(), viewModel.currentMode))
67  }
```

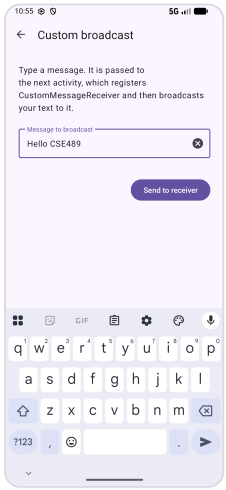
broadcast/BroadcastInputActivity.kt lines 138–141

```
138     /** Explicit Intent that opens this activity in the given [mode]. */
139     fun newIntent(context: Context, mode: BroadcastMode): Intent =
140         Intent(context, BroadcastInputActivity::class.java).putExtra(EXTRA_MODE, mode.name)
141 }
```

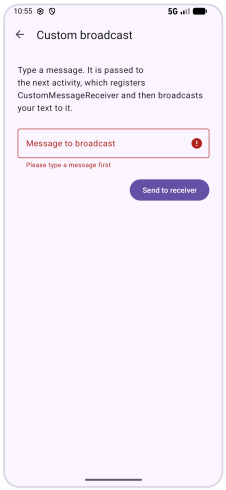
PUB/SUB & MESSAGE-QUEUE VIEW

- The Spinner does not change the description directly. It sends the choice to the ViewModel, which publishes it through `selectedMode`; the description is one subscriber. This one-way loop (event → state → screen) is called **unidirectional data flow**.
- The mode travels in the Intent as its **name** (`"CUSTOM"`), a simple string that survives being stored and copied by the system. This is the **message format** both sides agree on.
- The receiving activity parses it with `fromName()`, which returns `null` for anything unknown, and then closes itself instead of crashing. Rejecting a malformed message safely is what messaging systems call handling a **poison message**.

5.3 Activity 2 (custom): typing the message



Typing the message



Blank input is rejected

Activity 2 reads the mode from its Intent and shows one of two layouts (both are included in `activity_broadcast_input.xml` and hidden by default):

broadcast/BroadcastInputActivity.kt lines 42–50

```
42  when (BroadcastMode.fromName(intent.getStringExtra(EXTRA_MODE))) {
43      BroadcastMode.CUSTOM -> showCustomInput()
44      BroadcastMode.BATTERY -> showBatteryStatus()
45      null -> {
46          // Only BroadcastFragment starts this activity, and it always sends a mode.
47          Log.w(TAG, "Started without a valid broadcast mode; closing")
48          finish()
49      }
50  }
```

In custom mode it wires the text field, the keyboard's Send key and the button to one function:

broadcast/BroadcastInputActivity.kt lines 55–90

```
55  private fun showCustomInput() {
56      binding.toolbar.setTitle(R.string.title_custom_input)
57      val content = binding.customInput
58      content.root.isVisible = true
59
60      // Clear the "blank message" error as soon as the user types again.
61      content.messageInput.doAfterTextChanged { content.messageLayout.error = null }
62      // The keyboard's Send action does the same as the button.
63      content.messageInput.setOnEditorActionListener { _, actionId, event ->
64          when {
65              actionId == EditorInfo.IME_ACTION_SEND -> {
66                  sendMessage()
67                  true
68              }
69              // A hardware keyboard's Enter key arrives as IME_NULL plus the key event
70              // (once for key down, once for key up): send once, consume both.
71              actionId == EditorInfo.IME_NULL && event?.keyCode == KeyEvent.KEYCODE_ENTER -> {
72                  if (event.action == KeyEvent.ACTION_DOWN) sendMessage()
73                  true
74              }
75              else -> false
76          }
77      }
78      content.sendButton.setOnClickListener { sendMessage() }
79  }
80
81  /** Opens Activity 3 with the typed text, or shows an inline error if the text is blank. */
82  private fun sendMessage() {
83      val content = binding.customInput
84      val message = content.messageInput.text?.toString().orEmpty().trim()
85      if (message.isEmpty()) {
86          content.messageLayout.error = getString(R.string.error_blank_message)
87          return
88      }
89      startActivity(CustomReceiverActivity.newIntent(this, BroadcastMode.CUSTOM, message))
90  }
```

1. `sendMessage()` trims the text. If it is empty, it shows an **inline error** on the `TextInputLayout` and stops.
2. `doAfterTextChanged` removes that error as soon as the user types again.

3. The keyboard's *Send* key (`IME_ACTION_SEND`) calls the same function. A hardware keyboard's Enter arrives twice (key down and key up), so it sends only on `ACTION_DOWN` .
4. For valid text it starts Activity 3 with `mode = CUSTOM` **and the text inside the Intent**. It does **not** send a broadcast. Section 5.6 explains why.

PUB/SUB & MESSAGE-QUEUE VIEW

Activity 2 is a **producer that validates before it sends**: an empty message never enters the system. The text then travels **point-to-point** inside the Intent that starts Activity 3. That Intent is a reliable channel: the system always delivers it to the activity it names, and keeps it with that activity, so it is still there after a rotation.

5.4 CustomMessageReceiver: the subscriber

This is the custom `BroadcastReceiver` the brief asks for. It is deliberately small:

broadcast/CustomMessageReceiver.kt lines 19–51

```kotlin
class CustomMessageReceiver(
    private val onMessage: (String) -> Unit,
) : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action != ACTION_CUSTOM_MESSAGE) return
        val message = intent.getStringExtra(EXTRA_MESSAGE) ?: return
        Log.i(TAG, "Received via broadcast: $message")
        onMessage(message)
    }

    companion object {
        private const val TAG = "CustomMessageReceiver"

        /** Our own action. The application ID prefix keeps it unique on the device. */
        const val ACTION_CUSTOM_MESSAGE = "${BuildConfig.APPLICATION_ID}.action.CUSTOM_MESSAGE"

        /** The extra that carries the user's text. */
        const val EXTRA_MESSAGE = "${BuildConfig.APPLICATION_ID}.extra.MESSAGE"

        /** The filter the receiver is registered with: it only matches our custom action. */
        fun intentFilter(): IntentFilter = IntentFilter(ACTION_CUSTOM_MESSAGE)

        /**
         * Builds the broadcast for [message]. setPackage() restricts delivery to this app, so no
         * other app can receive the user's text.
         */
        fun createBroadcast(context: Context, message: String): Intent =
            Intent(ACTION_CUSTOM_MESSAGE)
                .setPackage(context.packageName)
                .putExtra(EXTRA_MESSAGE, message)
    }
}
```

Part	What it does	Messaging name
`onReceive()`	Checks the action and that the text is present, logs it, passes it to the `onMessage` callback	Consumer with message validation

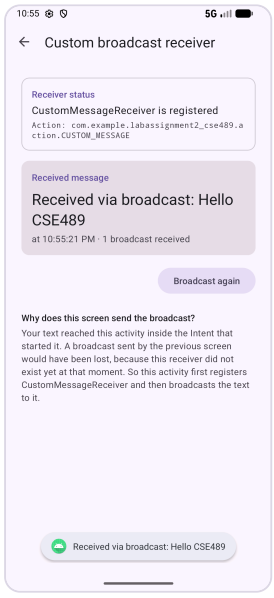
Part	What it does	Messaging name
`onMessage: (String) -> Unit`	A function given by Activity 3. The receiver knows nothing about screens; it only hands the text on	Keeps the consumer decoupled from the UI
`ACTION_CUSTOM_MESSAGE`	`com.example.labassignment2_cse489.action.CUSTOM_MESSAGE`: prefixed with the app ID so no other app's action can clash	A namespaced **topic name**
`EXTRA_MESSAGE`	The key under which the text travels	The **payload** field
`intentFilter()`	Matches only our action	The **subscription filter**
`createBroadcast()`	Builds the Intent, restricted to this app with `setPackage`	Message builder with an audience limit

Putting the action, the extra key, the filter and the message builder together in one `companion object` means the publisher and the subscriber can never disagree about the topic name or the payload format.

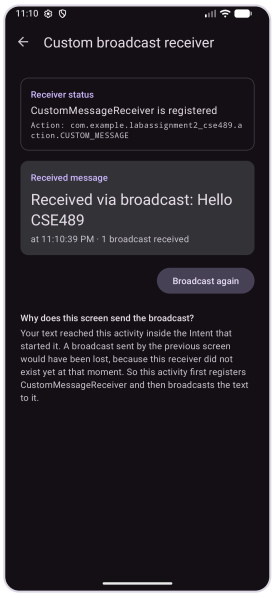
COMMON MISTAKE

Doing slow work in `onReceive()`. It runs on the main thread (section 3.3), and the system gives it only a few seconds before reporting the app as not responding. This receiver just checks, logs and hands over: it finishes almost instantly.

5.5 Activity 3: subscribe, publish, receive



Received through the broadcast



The same screen in dark theme

The brief says: "in this activity *create* the custom broadcast receiver which will receive the text message given in the second activity". Here is how Activity 3 does it.

Step 1 · Create the receiver (`onCreate`)

broadcast/CustomReceiverActivity.kt lines 64–77

```
64  private fun showCustomReceiver(text: String) {
65      message = text
66      binding.toolbar.setTitle(R.string.title_custom_receiver)
67      val content = binding.customReceiver
68      content.root.isVisible = true
69      content.receiverAction.text =
70          getString(R.string.receiver_action, CustomMessageReceiver.ACTION_CUSTOM_MESSAGE)
71      content.broadcastAgainButton.setOnClickListener { sendCustomBroadcast() }
72
73      // The brief asks for the custom receiver to be created here, in Activity 3.
74      receiver = CustomMessageReceiver { received -> onBroadcastReceived(received) }
75
76      viewModel.lastReceived.observe(this) { received -> renderReceived(received) }
77  }
```

The receiver is created with a callback that points back to the activity, and the activity starts observing the ViewModel's `lastReceived` LiveData (what was received so far).

Step 2 · Subscribe, then publish (`onStart`)

broadcast/CustomReceiverActivity.kt lines 79–103

```
79  override fun onStart() {
80      super.onStart()
81      val receiver = receiver ?: return // battery mode: nothing to register
82
83      // Android 14+ requires an export flag when registering a receiver for a non-system
84      // broadcast. RECEIVER_NOT_EXPORTED means only this app can send to it.
85      ContextCompat.registerReceiver(
86          this,
87          receiver,
88          CustomMessageReceiver.intentFilter(),
89          ContextCompat.RECEIVER_NOT_EXPORTED,
90      )
91      renderRegistration(registered = true)
92
93      // Why does Activity 3 send the broadcast itself? The text already reached this activity
94      // inside the Intent that started it. Had Activity 2 broadcast it before starting us, the
95      // broadcast would have been lost: at that moment this receiver did not exist yet, and a
96      // normal (non-sticky) broadcast is never kept for receivers that register later. So we
97      // register first and then broadcast the text to our own receiver.
98      // The ViewModel remembers that it was sent, so a rotation does not send it again.
99      if (!viewModel.initialBroadcastSent) {
100         viewModel.initialBroadcastSent = true
101         sendCustomBroadcast()
102     }
103 }
```

Step 3 · Receive and show (the callback)

broadcast/CustomReceiverActivity.kt lines 114–139

```
114 private fun sendCustomBroadcast() {
115     val text = message ?: return
116     sendBroadcast(CustomMessageReceiver.createBroadcast(this, text))
117 }
```

```
118
119 private fun onBroadcastReceived(text: String) {
120     viewModel.onMessageReceived(text, System.currentTimeMillis())
121     Toast.makeText(this, getString(R.string.received_toast, text), Toast.LENGTH_SHORT).show()
122 }
123
124 private fun renderReceived(received: ReceivedMessage?) {
125     val content = binding.customReceiver
126     if (received == null) {
127         content.receivedMessage.setText(R.string.received_waiting)
128         content.receivedMeta.isVisible = false
129     } else {
130         content.receivedMessage.text = getString(R.string.received_message, received.text)
131         content.receivedMeta.isVisible = true
132         content.receivedMeta.text = resources.getQuantityString(
133             R.plurals.received_meta,
134             received.count,
135             formatTimeOfDay(received.receivedAtMillis),
136             received.count,
137         )
138     }
139 }
```

Step 4 · Unsubscribe (`onStop`)

broadcast/CustomReceiverActivity.kt lines 105–112

```
105 override fun onStop() {
106     // Unregister in the callback that matches onStart, so the receiver can never leak.
107     receiver?.let { registered ->
108         unregisterReceiver(registered)
109         renderRegistration(registered = false)
110     }
111     super.onStop()
112 }
```

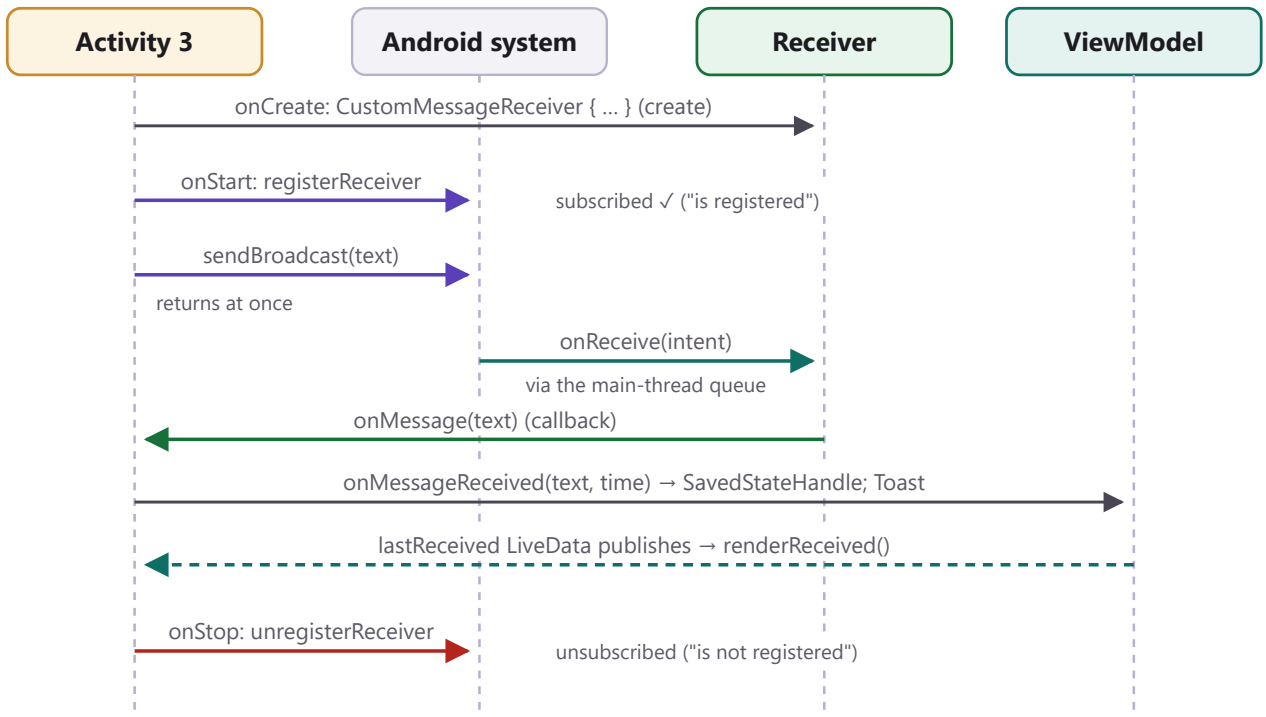
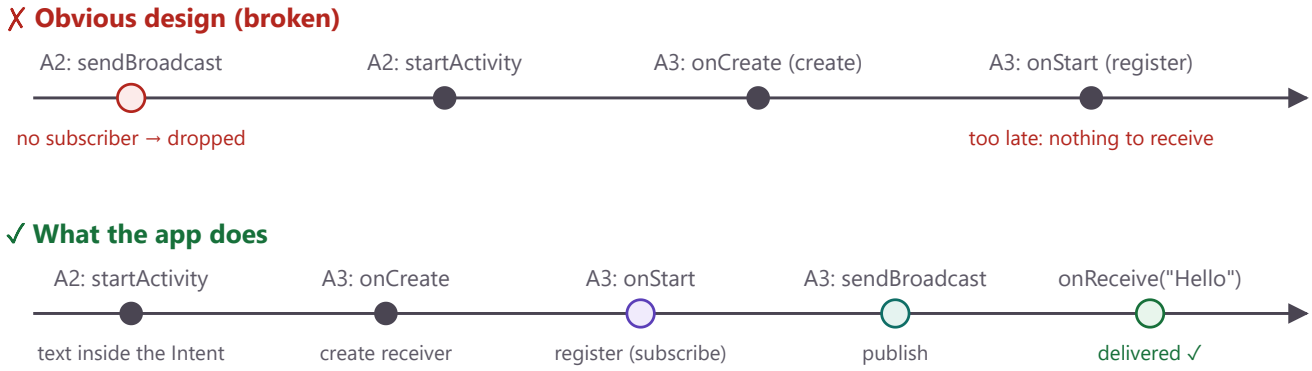


Figure 5.2 — The life of one custom broadcast, from subscribing to unsubscribing.

Broadcast again simply calls `sendCustomBroadcast()` once more: a new message on the same topic. The counter ("2 broadcasts received") proves each publish produced exactly one delivery to our one subscriber. The receiver also writes `Received via broadcast: …` to Logcat (`adb logcat -s CustomMessageReceiver`).

5.6 The lost-message problem

The obvious design would be: Activity 2 calls `sendBroadcast(text)`, then opens Activity 3, whose receiver picks it up. That **does not work**, and the reason is pure messaging theory:



Rule: a normal broadcast reaches only receivers that are registered at the moment it is sent.

Figure 5.3 — Subscribe first, publish second.

A normal broadcast is **non-durable** and **at-most-once** (section 2.5): it is delivered to the receivers registered *at that moment* and then forgotten. When Activity 2 would send it, the receiver does not exist yet; it is only created in Activity 3's `onCreate`. There is no **time decoupling**, so the message is lost.

The app's solution is a classic **store-and-forward**: the text is first carried **point-to-point** inside the Intent (a channel that does not lose it), and Activity 3 **re-publishes** it on the topic once its subscriber is in place. The code comment says exactly this:

broadcast/CustomReceiverActivity.kt lines 93–102

```
93  // Why does Activity 3 send the broadcast itself? The text already reached this activity
94  // inside the Intent that started it. Had Activity 2 broadcast it before starting us, the
95  // broadcast would have been lost: at that moment this receiver did not exist yet, and a
96  // normal (non-sticky) broadcast is never kept for receivers that register later. So we
97  // register first and then broadcast the text to our own receiver.
98  // The ViewModel remembers that it was sent, so a rotation does not send it again.
99  if (!viewModel.initialBroadcastSent) {
100     viewModel.initialBroadcastSent = true
101     sendCustomBroadcast()
102 }
```

Alternatives that were rejected

Idea	Why it is not used
Our own **sticky** broadcast (`sendStickyBroadcast`)	Would retain the message for late subscribers, but it has been deprecated since Android 5.0 (API 21) and is insecure: any app can read or replace a sticky broadcast.
A **manifest** `<receiver>`	The brief wants the receiver *created in Activity 3*; a manifest receiver is created by the system instead.

Idea	Why it is not used
Send from Activity 2 **after a short delay**	A race: it works only if Activity 3 happens to register in time. Message ordering must not depend on luck.
`LocalBroadcastManager`	Deprecated, and it is an in-app event bus, not a real broadcast.

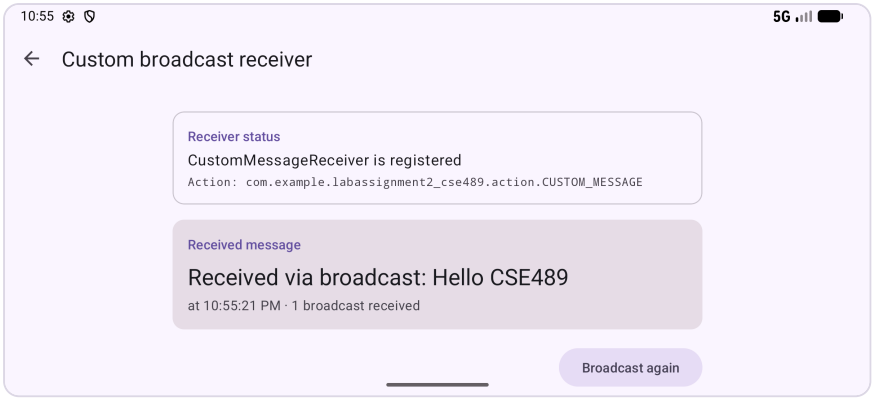
5.7 Rotation: no duplicates, nothing lost

Rotating the phone destroys Activity 3 and creates a new one. The old one runs `onStop` (unsubscribe); the new one runs `onCreate` and `onStart` again (subscribe). Two problems appear, and `CustomReceiverViewModel` solves both:

broadcast/CustomReceiverViewModel.kt lines 16–38, 45–49

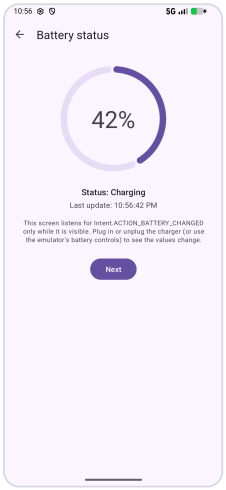
```
16  class CustomReceiverViewModel(private val savedStateHandle: SavedStateHandle) : ViewModel() {
17
18      /** The last message received, or null while waiting for the first broadcast. */
19      val lastReceived: LiveData<ReceivedMessage?> = savedStateHandle.getLiveData(KEY_RECEIVED, null)
20
21      /** True once Activity 3 has sent its first broadcast. */
22      var initialBroadcastSent: Boolean
23          get() = savedStateHandle[KEY_INITIAL_SENT] ?: false
24          set(value) {
25              savedStateHandle[KEY_INITIAL_SENT] = value
26          }
27
28      /** Records a delivery of the custom broadcast. */
29      fun onMessageReceived(text: String, receivedAtMillis: Long) {
30          val count = (lastReceived.value?.count ?: 0) + 1
31          savedStateHandle[KEY_RECEIVED] = ReceivedMessage(text, receivedAtMillis, count)
32      }
33
34      private companion object {
35          const val KEY_RECEIVED = "last_received"
36          const val KEY_INITIAL_SENT = "initial_broadcast_sent"
37      }
38  }
    ⋮
45  data class ReceivedMessage(
46      val text: String,
47      val receivedAtMillis: Long,
48      val count: Int,
49  ) : Serializable // so the SavedStateHandle can put it in a Bundle
```

- **Duplicate publish.** `onStart` would send the broadcast again, and the user would see a second "received". The `initialBroadcastSent` flag remembers that the first broadcast was already published, so only the first `onStart` ever sends it. In messaging terms this is **de-duplication on the producer side**, which turns an "at-least-once" risk into "exactly once" for this message.
- **Lost display.** The new activity's views are empty. The last received message, its time and the count live in `lastReceived`; the new activity observes it in `onCreate` and LiveData immediately delivers the stored value (a retained value, Figure 2.4).
- Both live in the `SavedStateHandle`, so even if Android kills the process in the background, nothing is re-sent and nothing is forgotten. `ReceivedMessage` is `Serializable` so that it can be stored in that Bundle.

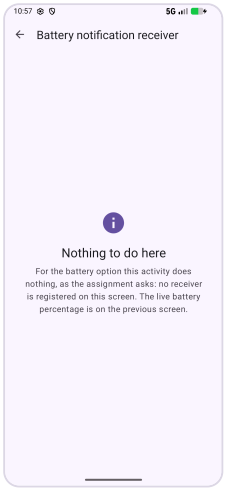


After rotating: same message, still "1 broadcast received"

5.8 Battery: a system topic with a retained message



Activity 2: live battery status



Activity 3: nothing to do

For the second Spinner option the app **subscribes** to a topic that Android itself publishes: `Intent.ACTION_BATTERY_CHANGED`. Three facts about this broadcast drive the whole design:

Fact	Meaning	Consequence in the code
Sticky	The system keeps the last battery Intent; `registerReceiver()` *returns* it at once	The percentage appears immediately, without waiting for the battery to change
Context-registered only	It is never delivered to a receiver declared in the manifest	The receiver is registered in Kotlin code
Protected	Only the system may send it	No export flag is needed when registering (5.11)

In Part 2 terms it is a **retained topic**: the battery service publishes a new message whenever the level, plug or status changes, and the broker keeps the latest one for every new subscriber.

Activity 2 shows the battery layout and simply **observes** a LiveData. It never registers a receiver itself:

broadcast/BroadcastInputActivity.kt lines 30–31, 94–120

```
30  // Created lazily, so in custom mode no battery LiveData (and no receiver) ever exists.
31  private val batteryViewModel: BatteryViewModel by viewModels()
    ⋮
```

```
94  private fun showBatteryStatus() {
95      binding.toolbar.setTitle(R.string.title_battery_status)
96      val content = binding.batteryStatus
97      content.root.isVisible = true
98      content.nextButton.setOnClickListener {
99          startActivity(CustomReceiverActivity.newIntent(this, BroadcastMode.BATTERY))
100     }
101
102     // Observing with this activity as the LifecycleOwner makes the LiveData active (it
103     // registers its receiver) in onStart and inactive (it unregisters) in onStop.
104     batteryViewModel.batteryStatus.observe(this) { status -> renderBattery(status) }
105 }
106
107 private fun renderBattery(status: BatteryStatus) {
108     val content = binding.batteryStatus
109     val percent = status.percent
110     content.batteryPercent.text = if (percent != null) {
111         getString(R.string.battery_percent, percent)
112     } else {
113         getString(R.string.battery_percent_unknown)
114     }
115     content.batteryIndicator.setProgressCompat(percent ?: 0, true)
116     content.batteryStatusText.text =
117         getString(R.string.battery_status_label, getString(status.chargingStatus.labelRes))
118     content.batteryUpdated.text =
119         getString(R.string.battery_last_update, formatTimeOfDay(status.receivedAtMillis))
120 }
```

broadcast/BatteryViewModel.kt lines 14–17

```
14  class BatteryViewModel(application: Application) : AndroidViewModel(application) {
15
16      val batteryStatus: LiveData<BatteryStatus> = BatteryStatusLiveData(application)
17  }
```

Next opens Activity 3 in battery mode, which, as the brief says, does nothing. It registers no receiver (`receiver` stays `null`, so `onStart` returns early) and only explains that:

broadcast/CustomReceiverActivity.kt lines 147–150

```
147 private fun showNothingToDo() {
148     binding.toolbar.setTitle(R.string.title_battery_receiver)
149     binding.batteryNothing.root.isVisible = true
150 }
```

5.9 BatteryStatusLiveData: subscribing only while visible

This small class is the cleverest part of Feature A. It is a `LiveData` that **owns** the battery receiver and registers it only while someone is watching:

broadcast/BatteryStatusLiveData.kt lines 17–45

```
17  class BatteryStatusLiveData(context: Context) : LiveData<BatteryStatus>() {
18
19      // The application context outlives every activity, so keeping it here cannot leak a screen.
20      private val appContext = context.applicationContext
21
22      private val receiver = object : BroadcastReceiver() {
23          override fun onReceive(context: Context, intent: Intent) {
```

```
24              // Context-registered receivers run on the main thread, so setValue() is safe here.
25              value = BatteryStatus.fromIntent(intent)
26          }
27      }
28
29      override fun onActive() {
30          // ACTION_BATTERY_CHANGED can only be received by a receiver registered in code (not in
31          // the manifest). It is a protected broadcast that only the system may send, so it is
32          // exempt from the Android 14 rule that registerReceiver() must pass RECEIVER_EXPORTED or
33          // RECEIVER_NOT_EXPORTED.
34          //
35          // It is also *sticky*: the system keeps the last one, and registerReceiver() returns it
36          // straight away, so the screen shows a value before the battery next changes.
37          val stickyIntent: Intent? =
38              appContext.registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_CHANGED))
39          stickyIntent?.let { value = BatteryStatus.fromIntent(it) }
40      }
41
42      override fun onInactive() {
43          appContext.unregisterReceiver(receiver)
44      }
45  }
```

1. LiveData calls `onActive()` when it gets its **first active observer**: an observer whose lifecycle is at least STARTED. Activity 2 observes with itself as the owner, so this happens in Activity 2's `onStart`.
2. `onActive()` registers the receiver. Because the broadcast is sticky, the call returns the latest battery Intent, which becomes the first `value` right away.
3. Each later battery broadcast calls `onReceive()` on the main thread, which sets a new `value`; LiveData passes it to the observer, which calls `renderBattery()`.
4. When Activity 2 stops (Home, Next, rotation), the observer becomes inactive, LiveData calls `onInactive()`, and the receiver is unregistered. It can never leak.
5. It keeps only the **application** context, which lives as long as the app, so it can never hold on to a destroyed activity.

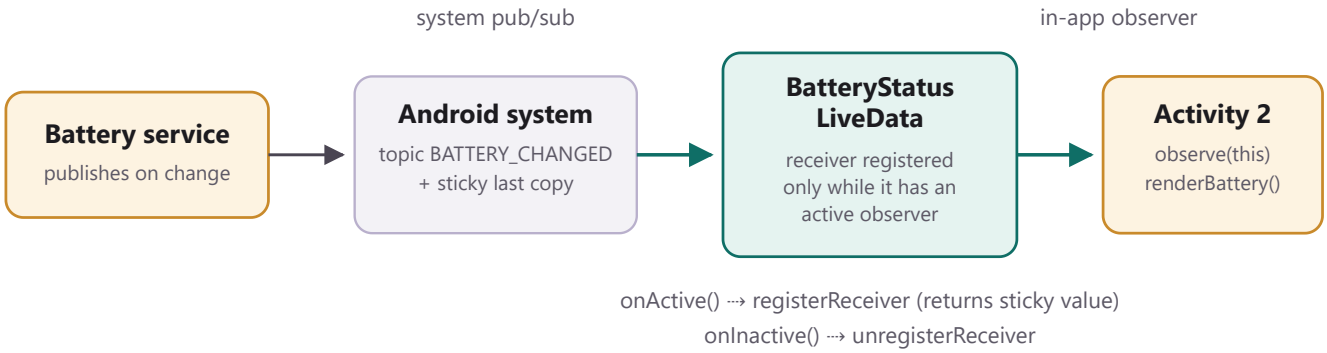


Figure 5.4 — BatteryStatusLiveData bridges the system's pub/sub topic and the app's lifecycle-aware observer.

PUB/SUB & MESSAGE-QUEUE VIEW

- `BatteryStatusLiveData` is a **bridge** between two messaging systems: the Android broadcast system (cross-process pub/sub) and LiveData (an in-process observable).
- It subscribes **upstream** (to the system) only while it has **downstream** subscribers (active observers). Reactive libraries call this a *reference-counted* subscription (Kotlin Flow's `WhileSubscribed`, RxJava's `refCount`). It saves battery and makes leaks impossible.
- Both ends retain the latest value: the system's sticky copy feeds the LiveData the moment it subscribes, and the LiveData's own value feeds a **new** activity after a rotation, because `BatteryViewModel` keeps the same LiveData object.
- The battery is a **state**, not an event: only the latest reading matters, so conflation (dropping old readings) is correct.
- In custom mode the `BatteryViewModel` is never even created (`by viewModels()` is lazy), so no battery subscription exists at all.

5.10 Turning the battery Intent into a percentage

The battery Intent does not contain "42 %". It contains `EXTRA_LEVEL` (for example 42) and `EXTRA_SCALE` (the maximum, usually 100, but some devices use other values). The app turns these raw extras into its own small model, `BatteryStatus`:

broadcast/BatteryStatus.kt lines 15–49

```
15  data class BatteryStatus(
16      val percent: Int?,
17      val chargingStatus: ChargingStatus,
18      val receivedAtMillis: Long,
19  ) {
20      companion object {
21          /** Value returned by getIntExtra() when the battery intent lacks an extra. */
22          private const val EXTRA_MISSING = -1
23          private const val PERCENT_MAX = 100
24
25          /**
26           * Battery percentage = EXTRA_LEVEL * 100 / EXTRA_SCALE.
27           *
28           * Returns null when the level is missing (negative) or the scale is not positive, which
29           * would otherwise be a division by zero. The result is clamped to 0..100.
30           */
31          fun percentOf(level: Int, scale: Int): Int? {
32              if (level < 0 || scale <= 0) return null
33              return (level * PERCENT_MAX / scale).coerceIn(0, PERCENT_MAX)
34          }
35
36          /** Reads the extras of an ACTION_BATTERY_CHANGED [intent]. */
37          fun fromIntent(
38              intent: Intent,
39              receivedAtMillis: Long = System.currentTimeMillis(),
40          ): BatteryStatus = BatteryStatus(
41              percent = percentOf(
42                  level = intent.getIntExtra(BatteryManager.EXTRA_LEVEL, EXTRA_MISSING),
43                  scale = intent.getIntExtra(BatteryManager.EXTRA_SCALE, EXTRA_MISSING),
44              ),
45              chargingStatus = ChargingStatus.fromExtra(
46                  intent.getIntExtra(BatteryManager.EXTRA_STATUS, BatteryManager.BATTERY_STATUS_UNKNOWN),
```

```
            ),
            receivedAtMillis = receivedAtMillis,
        )
```

- **Percentage** = `level × 100 / scale`, clamped to 0…100.
- If the level is missing, `getIntExtra` returns the default `-1`; if the scale is 0, dividing would crash. Both cases give `null`, which the screen shows as "—".
- `EXTRA_STATUS` is mapped to the `ChargingStatus` enum, each value carrying its on-screen label. Anything unexpected becomes `UNKNOWN`:

broadcast/BatteryStatus.kt lines 53–70

```
/** The BatteryManager.EXTRA_STATUS values, each with the label shown on screen. */
enum class ChargingStatus(@StringRes val labelRes: Int) {
    CHARGING(R.string.battery_status_charging),
    DISCHARGING(R.string.battery_status_discharging),
    FULL(R.string.battery_status_full),
    NOT_CHARGING(R.string.battery_status_not_charging),
    UNKNOWN(R.string.battery_status_unknown);

    companion object {
        /** Maps a BatteryManager.BATTERY_STATUS_* constant; anything unexpected is [UNKNOWN]. */
        fun fromExtra(status: Int): ChargingStatus = when (status) {
            BatteryManager.BATTERY_STATUS_CHARGING -> CHARGING
            BatteryManager.BATTERY_STATUS_DISCHARGING -> DISCHARGING
            BatteryManager.BATTERY_STATUS_FULL -> FULL
            BatteryManager.BATTERY_STATUS_NOT_CHARGING -> NOT_CHARGING
            else -> UNKNOWN
        }
    }
```

`receivedAtMillis` records when the message was read; the screen shows it as "Last update: 10:56:42 PM" so you can see live updates arriving. All of this maths is plain Kotlin and is unit-tested (Part 10).

PUB/SUB & MESSAGE-QUEUE VIEW

The Intent extras are the **message schema** defined by the publisher (Android). A good consumer immediately translates foreign messages into its own model and treats every field as possibly missing or invalid. Doing that in one place (`fromIntent`) keeps the rest of the app free of raw Intent handling.

5.11 Who may publish and who may listen

A topic that any app can publish to, or listen on, is a security hole. The app closes every door:

Protection	What it prevents	Where
`RECEIVER_NOT_EXPORTED` when registering	Other apps **publishing** fake messages to our receiver. Since Android 14 (targetSdk 34+), a context-registered receiver for a non-system broadcast *must* say exported or not-exported, or `registerReceiver` throws a `SecurityException`	`CustomReceiverActivity.onStart`

Protection	What it prevents	Where
`ContextCompat.registerReceiver`	Works on old Android versions too, where it emulates "not exported" with a permission only this app holds	Same call
`setPackage(packageName)` on the broadcast	Other apps **listening** to the user's text	`CustomMessageReceiver.createBroadcast`
App-ID prefix on the action	Accidental clashes with another app's topic name	`ACTION_CUSTOM_MESSAGE`
Protected system broadcast	Fake battery messages: only the system may send `ACTION_BATTERY_CHANGED`, so the battery receiver needs no flag	`BatteryStatusLiveData.onActive`
`android:exported="false"` on Activities 2 and 3	Other apps starting them with crafted Intents (point-to-point access control)	`AndroidManifest.xml`

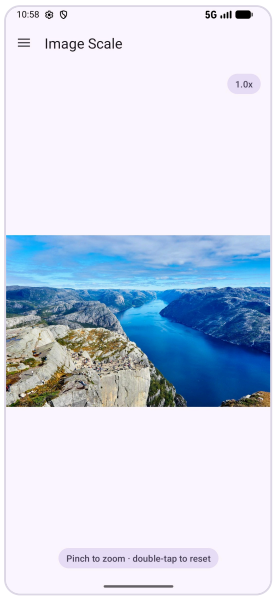
REMEMBER

In a broker, access control has two sides: **who may publish** to a topic and **who may subscribe** to it. Here `RECEIVER_NOT_EXPORTED` guards the subscriber (only trusted publishers), `setPackage` guards the message (only trusted subscribers), and a protected broadcast guards the topic itself (only the system publishes).

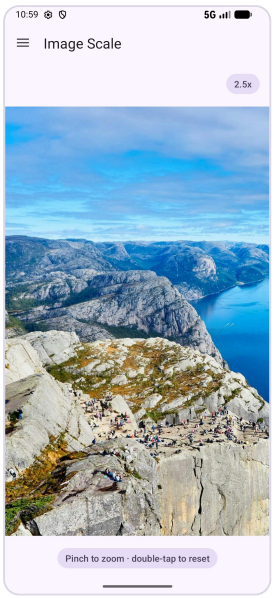
PART 6

Feature B: Image Scale

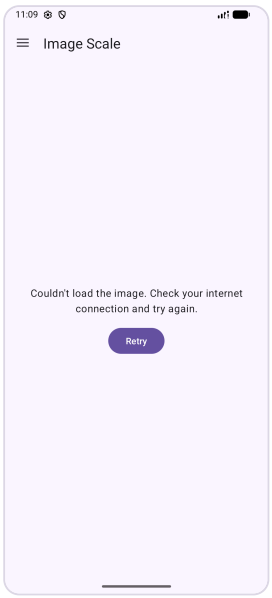
The brief: "Load an image from the internet and scale it with a pinch gesture." The app downloads a large photo and implements pinch-to-zoom itself, with no zoom library, using `ScaleGestureDetector` and an image `Matrix`.



At 1.0x (the whole photo fits)



Zoomed to 2.5x and panned



Offline: error with Retry

6.1 Downloading the picture with Coil

The screen can be in one of three states, written as a **sealed interface** (a closed list of possibilities that the compiler knows about):

image/ImageScaleViewModel.kt lines 19–24

```kotlin
/** What the Image Scale screen shows. */
sealed interface ImageUiState {
    data object Loading : ImageUiState
    data class Success(val image: Image) : ImageUiState
    data object Error : ImageUiState
}
```

The ViewModel starts the download as soon as it is created and publishes the result as a `StateFlow`:

image/ImageScaleViewModel.kt lines 33–68

```kotlin
class ImageScaleViewModel(application: Application) : AndroidViewModel(application) {

    private val _uiState = MutableStateFlow<ImageUiState>(ImageUiState.Loading)
    val uiState: StateFlow<ImageUiState> = _uiState.asStateFlow()

    private var loadJob: Job? = null

    init {
        load()
    }

```

```
    /** Starts (or restarts, e.g. from the Retry button) loading the image. */
    fun load() {
        loadJob?.cancel()
        _uiState.value = ImageUiState.Loading
        loadJob = viewModelScope.launch {
            val context = getApplication<Application>()
            val request = ImageRequest.Builder(context)
                .data(IMAGE_URL)
                // Coil normally decodes images at the size of their target view. Zooming into
                // such a downsampled bitmap looks blurry, so ask for the original resolution.
                .size(Size.ORIGINAL)
                .build()
            _uiState.value = when (val result = context.imageLoader.execute(request)) {
                is SuccessResult -> ImageUiState.Success(result.image)
                is ErrorResult -> {
                    Log.w(TAG, "Could not load $IMAGE_URL", result.throwable)
                    ImageUiState.Error
                }
            }
        }
    }

    companion object {
        /** A free 2000 x 1300 photo from Lorem Picsum (picsum.photos). */
        const val IMAGE_URL = "https://picsum.photos/id/1015/2000/1300"
```

- **Coil 3** loads images. Its core cannot download by itself; the `coil-network-okhttp` dependency adds the HTTP fetcher. The `INTERNET` permission is in the manifest.
- `imageLoader.execute(request)` is a **suspend** function: the coroutine pauses while Coil downloads and decodes on background threads, then continues on the main thread with the result. The screen never freezes.
- `.size(Size.ORIGINAL)` asks for the full 2000 × 1300 bitmap. By default Coil decodes at the size of the view, and zooming into that would look blurry.
- *Retry* calls `load()` again, which **cancels** any download still running (`loadJob?.cancel()`) and starts a fresh one.
- The download lives in the ViewModel, so a rotation does not restart it.

The fragment collects the state only while visible and draws it:

image/ImageScaleFragment.kt lines 36–60

```
    binding.zoomableImage.onZoomChanged = { zoom -> showZoom(zoom) }
    showZoom(binding.zoomableImage.zoom)
    binding.retryButton.setOnClickListener { viewModel.load() }

    // Collect the state only while the screen is visible: repeatOnLifecycle starts the block
    // in onStart and cancels it in onStop.
    viewLifecycleOwner.lifecycleScope.launch {
        viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
            viewModel.uiState.collect { state -> render(state) }
        }
    }
}

private fun render(state: ImageUiState) {
    binding.loadingIndicator.isVisible = state is ImageUiState.Loading
    binding.errorGroup.isVisible = state is ImageUiState.Error
    val loaded = state is ImageUiState.Success
```

```
53      binding.zoomLabel.isVisible = loaded
54      binding.zoomHint.isVisible = loaded
55
56      if (state is ImageUiState.Success && state.image !== shownImage) {
57          shownImage = state.image
58          binding.zoomableImage.setImageDrawable(state.image.asDrawable(resources))
59      }
60  }
```

The `state.image !== shownImage` check matters: every time the screen comes back to `onStart`, `repeatOnLifecycle` re-subscribes and the StateFlow re-delivers the current `Success`. Setting the same picture again would reset the zoom, so the fragment only sets a picture it has not shown yet.

PUB/SUB & MESSAGE-QUEUE VIEW

- The download is **asynchronous request/reply**: the request goes out, the coroutine waits without blocking the main thread, and the reply comes back as `SuccessResult` or `ErrorResult`.
- `uiState` is a **retained topic**. After a rotation the new fragment subscribes and receives `Success` immediately: no second download.
- A running `Job` is a **pending request**. Cancelling it on Retry is like withdrawing a message nobody needs any more, so an old, slow reply can never overwrite a newer one.

6.2 The image Matrix and the fit

With `scaleType = MATRIX`, an `ImageView` draws its picture through a `Matrix`: a small table of numbers that says how to move every pixel of the picture onto the screen. Here it only ever contains a **scale** (make bigger/smaller) and a **translation** (shift left/right/up/down). The app changes the matrix; the ImageView redraws.

image/ZoomableImageView.kt — lines 45–48, 70–76

```
45  private val drawMatrix = Matrix()
46  private val inverseMatrix = Matrix()
47  private val mappedRect = RectF()
48  private val point = FloatArray(2)
    ⋮
70  init {
71      scaleType = ScaleType.MATRIX
72      initialized = true
73  }
74
75  private val contentWidth get() = (width - paddingLeft - paddingRight).toFloat()
76  private val contentHeight get() = (height - paddingTop - paddingBottom).toFloat()
```

"1x" is the **fit**: the scale that shows the whole picture, centred, without stretching:

image/ZoomMath.kt lines 11–23

```kotlin
/** Zoom is relative to the fitted image: 1x shows the whole picture. */
const val MIN_ZOOM = 1f
const val MAX_ZOOM = 5f

/** Limits [zoom] to [MIN_ZOOM]..[MAX_ZOOM]. */
fun clampZoom(zoom: Float): Float = zoom.coerceIn(MIN_ZOOM, MAX_ZOOM)

/**
 * The scale that makes content of [contentWidth] x [contentHeight] fit entirely inside a
 * [viewWidth] x [viewHeight] view without distortion (like ScaleType.FIT_CENTER).
 */
fun fitScale(contentWidth: Float, contentHeight: Float, viewWidth: Float, viewHeight: Float): Float =
    min(viewWidth / contentWidth, viewHeight / contentHeight)
```

image/ZoomableImageView.kt lines 80–83, 104–118

```kotlin
override fun setImageDrawable(drawable: Drawable?) {
    super.setImageDrawable(drawable)
    if (initialized) refit(keep = null) // a new picture starts at "fit"
}
⋮
private fun refit(keep: ZoomState?) {
    val drawable = drawable ?: return
    val dw = drawable.intrinsicWidth.toFloat()
    val dh = drawable.intrinsicHeight.toFloat()
    if (dw <= 0f || dh <= 0f || contentWidth <= 0f || contentHeight <= 0f) return

    val fitScale = ZoomMath.fitScale(dw, dh, contentWidth, contentHeight)
    drawMatrix.setScale(fitScale, fitScale)
    drawMatrix.postTranslate((contentWidth - dw * fitScale) / 2f, (contentHeight - dh * fitScale) / 2f)
    zoom = ZoomMath.MIN_ZOOM

    val state = pendingState ?: keep
    pendingState = null
    if (state != null) restore(state) else applyMatrix()
}
```

A WORKED EXAMPLE

The photo is 2000 × 1300 px. On a phone the view might be 1080 × 1800 px. The fit scale is `min(1080 / 2000, 1800 / 1300) = min(0.54, 1.38) = 0.54`. So the photo is drawn 1080 × 702 px, and moved down by `(1800 − 702) / 2 = 549` px to centre it. That is "1.0x". "2.5x" then means 2.5 times the fit scale.

The fit is recomputed whenever the **picture** changes (`setImageDrawable`) or the **view size** changes (`onSizeChanged`, for example after rotation). This matters because Coil delivers the picture asynchronously, usually after the view already has its size, and `refit` does nothing until both are known.

6.3 Pinch zoom with ScaleGestureDetector

A pinch is just a stream of raw touch events from two fingers. `ScaleGestureDetector` turns that stream into a simple question-and-answer: "how much did the fingers spread, and around which point?"

- `focusX`, `focusY`: the **focal point**, half-way between the fingers.

- `scaleFactor`: current finger distance ÷ previous distance (for example 1.03 when the fingers move 3 % further apart since the last event).

image/ZoomableImageView.kt lines 58–62, 196–211

```
58  private val scaleDetector = ScaleGestureDetector(context, ScaleListener()).apply {
59      // Only the two-finger pinch zooms; "double-tap and drag" would fight the double tap.
60      isQuickScaleEnabled = false
61  }
62  private val gestureDetector = GestureDetector(context, GestureListener())
    ⋮
196 private inner class ScaleListener : ScaleGestureDetector.SimpleOnScaleGestureListener() {
197     override fun onScaleBegin(detector: ScaleGestureDetector): Boolean {
198         lastFocusX = detector.focusX
199         lastFocusY = detector.focusY
200         return true
201     }
202
203     override fun onScale(detector: ScaleGestureDetector): Boolean {
204         // Follow the fingers (two-finger pan), then scale around the pinch focal point.
205         drawMatrix.postTranslate(detector.focusX - lastFocusX, detector.focusY - lastFocusY)
206         lastFocusX = detector.focusX
207         lastFocusY = detector.focusY
208         zoomBy(detector.scaleFactor, detector.focusX, detector.focusY)
209         return true
210     }
211 }
```

In each `onScale` the picture first follows the movement of the focal point (so two fingers can also drag it), and then zooms around that point:

image/ZoomableImageView.kt lines 122–133

```
122 /**
123  * Multiplies the zoom by [factor] around the view point ([focusX], [focusY]), so the image
124  * point under the fingers stays under the fingers. The zoom is clamped to 1x..5x.
125  */
126 private fun zoomBy(factor: Float, focusX: Float, focusY: Float) {
127     val target = ZoomMath.clampZoom(zoom * factor)
128     val applied = target / zoom
129     drawMatrix.postScale(applied, applied, focusX - paddingLeft, focusY - paddingTop)
130     zoom = target
131     clampTranslation()
132     applyMatrix()
133 }
```

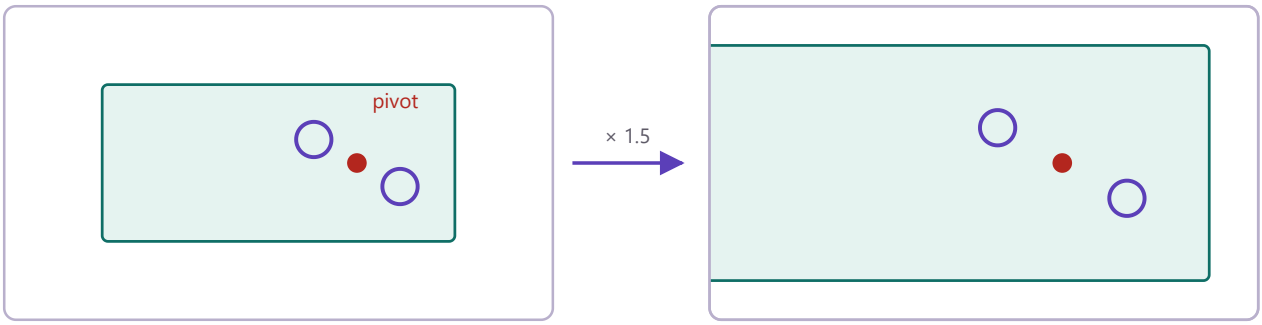


Figure 6.1 — `postScale(s, s, px, py)` moves every point p to px + s·(p − px). The pivot itself stays put, so the part of the picture under your fingers stays under your fingers.

- **Clamping first.** `target = clampZoom(zoom × factor)` keeps the zoom between 1x and 5x. Then `applied = target / zoom` is what is really left to apply. Example: at 4.5x a factor of 1.2 would give 5.4x; it is clamped to 5x, so only ×1.11 is applied.
- Quick-scale (double-tap-and-drag) is turned off, because it would fight with the double-tap gesture of the next section.

6.4 Pan, edges and double-tap

A second detector, `GestureDetector`, understands one-finger gestures:

image/ZoomableImageView.kt lines 213–233

```
213  private inner class GestureListener : GestureDetector.SimpleOnGestureListener() {
214      override fun onDown(e: MotionEvent): Boolean = true
215
216      override fun onScroll(e1: MotionEvent?, e2: MotionEvent, distanceX: Float, distanceY: Float):
     Boolean {
217          // One-finger pan only while zoomed in; at 1x the whole image is already visible.
218          if (scaleDetector.isInProgress || zoom <= ZoomMath.MIN_ZOOM) return false
219          panBy(-distanceX, -distanceY)
220          return true
221      }
222
223      override fun onDoubleTap(e: MotionEvent): Boolean {
224          val target = if (zoom > ZoomMath.MIN_ZOOM + ZOOM_EPSILON) ZoomMath.MIN_ZOOM else DOUBLE_TAP_ZOOM
225          animateZoomTo(target, e.x, e.y)
226          return true
227      }
228
229      override fun onSingleTapUp(e: MotionEvent): Boolean {
230          tapUpDetected = true
231          return false
232      }
233  }
```

- **Pan** (`onScroll`): move the picture by the finger's movement, but only while zoomed in. At 1x the whole picture is already visible.
- **Double-tap**: animate to 2.5x around the tapped point, or back to 1x if already zoomed.

After every move or zoom, the picture is pulled back so that no empty gap appears at an edge:

image/ZoomableImageView.kt lines 135–155

```kotlin
private fun panBy(dx: Float, dy: Float) {
    drawMatrix.postTranslate(dx, dy)
    clampTranslation()
    applyMatrix()
}

/** Moves the image back so it never leaves an empty gap at an edge it could cover. */
private fun clampTranslation() {
    val drawable = drawable ?: return
    mappedRect.set(0f, 0f, drawable.intrinsicWidth.toFloat(), drawable.intrinsicHeight.toFloat())
    drawMatrix.mapRect(mappedRect)
    drawMatrix.postTranslate(
        ZoomMath.edgeCorrection(mappedRect.left, mappedRect.right, contentWidth),
        ZoomMath.edgeCorrection(mappedRect.top, mappedRect.bottom, contentHeight),
    )
}

private fun applyMatrix() {
    imageMatrix = drawMatrix
    onZoomChanged?.invoke(zoom)
}
```

image/ZoomMath.kt lines 25–39

```kotlin
/**
 * How far content spanning [start]..[end] on one axis must move to sit correctly in a
 * viewport of [viewportSize]:
 *  - content smaller than the viewport is centred;
 *  - larger content is pulled back so no empty gap shows at either edge.
 */
fun edgeCorrection(start: Float, end: Float, viewportSize: Float): Float {
    val size = end - start
    return when {
        size <= viewportSize -> (viewportSize - size) / 2f - start
        start > 0f -> -start
        end < viewportSize -> viewportSize - end
        else -> 0f
    }
}
```

`mapRect` finds where the picture's rectangle is on screen right now. Then, separately for each axis: if the picture is **smaller** than the view it is **centred**; otherwise it is shifted just enough to close any gap at the start or end. The double-tap uses a `ValueAnimator` to zoom smoothly over 250 ms:

image/ZoomableImageView.kt lines 157–168

```kotlin
/** Smoothly zooms to [target] around ([focusX], [focusY]); used by the double tap. */
private fun animateZoomTo(target: Float, focusX: Float, focusY: Float) {
    zoomAnimator?.cancel()
    zoomAnimator = ValueAnimator.ofFloat(zoom, target).apply {
        duration = ZOOM_ANIMATION_MS
        interpolator = DecelerateInterpolator()
        addUpdateListener { animator ->
            zoomBy((animator.animatedValue as Float) / zoom, focusX, focusY)
        }
        start()
    }
}
```

Each animation frame calls `zoomBy` with "the zoom this frame should have ÷ the zoom we have now", so the same clamping and edge rules apply to the animation too.

6.5 Touch events are a message stream

Every finger contact is reported as a sequence of `MotionEvent` messages: `ACTION_DOWN` (first finger), `ACTION_POINTER_DOWN` (second finger), many `ACTION_MOVE`s, `ACTION_POINTER_UP` and `ACTION_UP`. They arrive one by one through the main thread's Looper (section 3.3) and are handed down the view tree: Activity → DrawerLayout → … → `ZoomableImageView.onTouchEvent`.

image/ZoomableImageView.kt lines 170–187

```
170 override fun onTouchEvent(event: MotionEvent): Boolean {
171     if (drawable == null) return super.onTouchEvent(event)
172     if (event.actionMasked == MotionEvent.ACTION_DOWN) zoomAnimator?.cancel()
173
174     scaleDetector.onTouchEvent(event)
175     gestureDetector.onTouchEvent(event)
176
177     // While zoomed (or pinching), don't let a parent such as the DrawerLayout steal the drag.
178     if (zoom > ZoomMath.MIN_ZOOM || event.pointerCount > 1) {
179         parent?.requestDisallowInterceptTouchEvent(true)
180     }
181     // A plain tap still counts as a click, which accessibility services rely on.
182     if (tapUpDetected) {
183         tapUpDetected = false
184         performClick()
185     }
186     return true
187 }
```

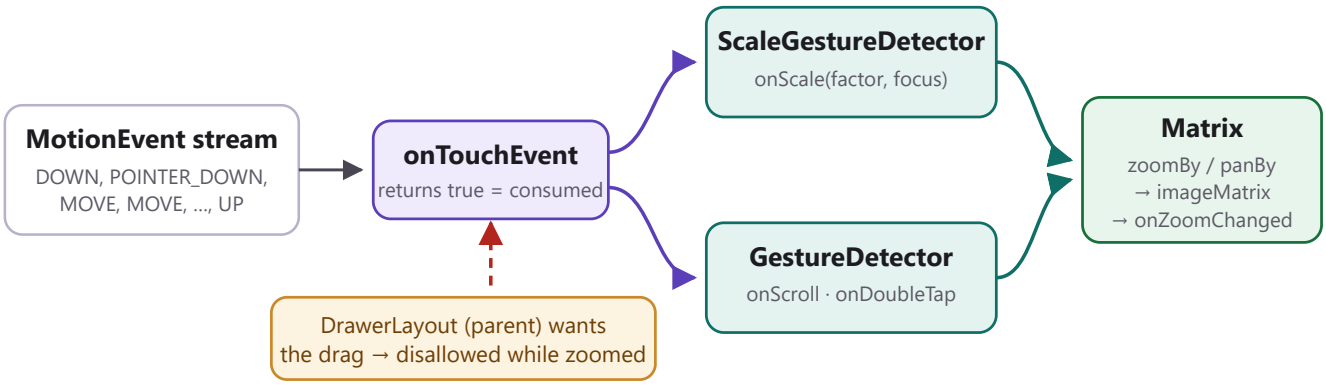


Figure 6.2 — One stream of touch messages, two stream processors, one result.

PUB/SUB & MESSAGE-QUEUE VIEW

- **Consuming a message.** Returning `true` from `onTouchEvent` means "this gesture is mine". Android then sends the rest of that finger's stream to this view only. Touch delivery is "one consumer wins", like a queue, not fan-out.
- **Stopping a competing consumer.** `DrawerLayout` watches touches too, because a drag from the left edge should open the drawer. While the picture is zoomed or pinched, the view calls `parent.requestDisallowInterceptTouchEvent(true)`: "parent, don't take these messages".
- **Fan-out to stream processors.** The same event is passed to *both* detectors. Each one keeps its own state and turns many low-level messages into a few high-level ones (`onScale`, `onScroll`, `onDoubleTap`). This is called *event stream processing*.
- **Timers are messages too.** To tell a single tap from a double tap, `GestureDetector` posts a *delayed* message to the main thread's queue and checks, when it arrives, whether a second tap came first.
- **Observer.** `onZoomChanged` is a one-subscriber callback: the view publishes every new zoom, and the fragment updates the "2.5x" label.

A plain tap still calls `performClick()`, which accessibility services such as TalkBack rely on.

6.6 Keeping the zoom through rotation

On rotation the old view is destroyed and a new, differently shaped view is created. Saving the raw matrix would be wrong, because the new view has a different size. Instead the view saves **what you were looking at**: the zoom, and which point of the picture was in the centre, as fractions (0…1) of the picture size.

image/ZoomableImageView.kt lines 237–256

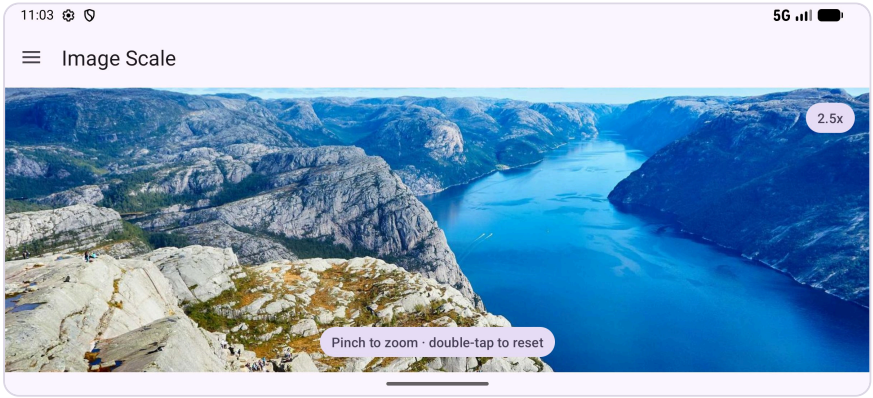
```
237 /**
238  * Zoom plus the image point at the centre of the view, as fractions (0..1) of the image
239  * size, so the same area can be shown again in a view of a different size.
240  */
241 private data class ZoomState(val zoom: Float, val centerX: Float, val centerY: Float)
242
243 private fun captureState(
244     viewportWidth: Float = contentWidth,
245     viewportHeight: Float = contentHeight,
246 ): ZoomState? {
247     val drawable = drawable ?: return null
248     val dw = drawable.intrinsicWidth.toFloat()
249     val dh = drawable.intrinsicHeight.toFloat()
250     if (dw <= 0f || dh <= 0f || viewportWidth <= 0f || viewportHeight <= 0f) return null
251     if (!drawMatrix.invert(inverseMatrix)) return null
252     point[0] = viewportWidth / 2f
253     point[1] = viewportHeight / 2f
254     inverseMatrix.mapPoints(point) // view centre -> drawable coordinates
255     return ZoomState(zoom, point[0] / dw, point[1] / dh)
256 }
```

image/ZoomableImageView.kt lines 258–291

```
258 /** Applies [state] on top of a freshly fitted matrix. */
259 private fun restore(state: ZoomState) {
260     val drawable = drawable ?: return
261     zoomBy(state.zoom / zoom, paddingLeft + contentWidth / 2f, paddingTop + contentHeight / 2f)
262     point[0] = state.centerX * drawable.intrinsicWidth
```

```
263     point[1] = state.centerY * drawable.intrinsicHeight
264     drawMatrix.mapPoints(point) // drawable point -> where it is in the view now
265     panBy(contentWidth / 2f - point[0], contentHeight / 2f - point[1])
266 }
267
268 override fun onSaveInstanceState(): Parcelable = Bundle().apply {
269     putParcelable(KEY_SUPER_STATE, super.onSaveInstanceState())
270     (pendingState ?: captureState())?.let { state ->
271         putFloat(KEY_ZOOM, state.zoom)
272         putFloat(KEY_CENTER_X, state.centerX)
273         putFloat(KEY_CENTER_Y, state.centerY)
274     }
275 }
276
277 override fun onRestoreInstanceState(state: Parcelable?) {
278     if (state !is Bundle) {
279         super.onRestoreInstanceState(state)
280         return
281     }
282     super.onRestoreInstanceState(BundleCompat.getParcelable(state, KEY_SUPER_STATE,
    Parcelable::class.java))
283     if (state.containsKey(KEY_ZOOM)) {
284         pendingState = ZoomState(
285             zoom = state.getFloat(KEY_ZOOM),
286             centerX = state.getFloat(KEY_CENTER_X),
287             centerY = state.getFloat(KEY_CENTER_Y),
288         )
289         refit(keep = null) // applies it now if the drawable and size are already known
290     }
291 }
```

1. `onSaveInstanceState`: invert the matrix to find which picture point is at the view centre, and store zoom + centre in a `Bundle`.
2. `onRestoreInstanceState` (new view): read them into `pendingState`. The picture and size may not be known yet, so it waits.
3. As soon as `refit()` has both a picture and a size, it applies the pending state: zoom by the saved factor, then pan so the saved picture point is back in the centre.



The same zoom and area after rotating to landscape

PUB/SUB & MESSAGE-QUEUE VIEW

The saved state is a **message from the old view to its future replacement**, stored by the system in between: store-and-forward once more. `pendingState` is a message **held until the consumer is ready** (deferred delivery): it is applied only when the view can actually use it.

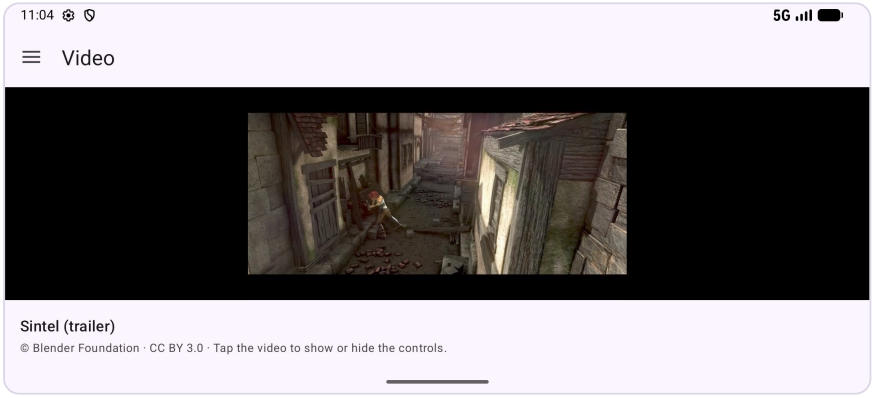
PART 7

Feature C: Video

The brief: "Play one video within app (media player)." The app plays the *Sintel* trailer (© Blender Foundation, CC BY 3.0), bundled as `res/raw/sample_video.mp4`, so it works offline.



Playing, with the control bar



Landscape: playback continued through the rotation

7.1 VideoView and MediaController

- **`VideoView`** is a ready-made view that shows video. Inside, it creates an `android.media.MediaPlayer` and draws the frames on a *surface*. It also keeps the video's aspect ratio inside its black frame.
- **`MediaController`** is the standard control bar: play/pause, a seek bar, and the current and total time. Tapping the video shows or hides it.

video/VideoFragment.kt lines 40–59

```kotlin
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    _binding = FragmentVideoBinding.bind(view)

    val controller = MediaController(requireContext())
    mediaController = controller

    binding.videoView.apply {
        // VideoView anchors the controller to its parent (the black frame), so the
        // controls appear along the bottom edge of the video area.
        setMediaController(controller)
        setOnPreparedListener { onVideoPrepared() }
        setOnErrorListener { _, what, extra -> onVideoError(what, extra) }
        setOnCompletionListener { onVideoCompleted() }
        setVideoURI("android.resource://${requireContext().packageName}/${R.raw.sample_video}".toUri())
    }

    drawerLayout = requireActivity().findViewById(R.id.drawer_layout)
    drawerLayout?.addDrawerListener(hideControlsWhileDrawerMoves)
}
```

The video is addressed with a resource URI, `android.resource://<package>/<resource id>`, which points into the app's own `res/raw` folder. Three listeners are registered before the video is opened: prepared, error and completed.

7.2 Pausing, remembering, resuming

A `VideoView` cannot remember anything across a rotation: it is destroyed with the fragment's view, and it even releases its player whenever the app goes to the background. So the position lives in a small ViewModel, backed by `SavedStateHandle`:

video/VideoViewModel.kt lines 14–31

```kotlin
class VideoViewModel(private val savedStateHandle: SavedStateHandle) : ViewModel() {

    /** Position in milliseconds to continue from. */
    val positionMs: Int
        get() = savedStateHandle[KEY_POSITION_MS] ?: 0

    /** Whether to start playing again as soon as the video is prepared. */
    val resumePlaying: Boolean
        get() = savedStateHandle[KEY_RESUME_PLAYING] ?: false

    /** Called when the screen stops. */
    fun savePlayback(positionMs: Int, resumePlaying: Boolean) {
        savedStateHandle[KEY_POSITION_MS] = positionMs
        savedStateHandle[KEY_RESUME_PLAYING] = resumePlaying
    }

    /** After the video ends, the next visit starts from the beginning. */
    fun onPlaybackCompleted() = savePlayback(positionMs = 0, resumePlaying = false)
```

The fragment saves the position in **`onPause`**:

video/VideoFragment.kt lines 61–89

```kotlin
override fun onStart() {
```

```
62      super.onStart()
63      // Back from the background the video is being prepared again.
64      if (!isPrepared) showLoading()
65  }
66
67  override fun onPause() {
68      super.onPause()
69      // The screen is about to stop (rotation, Home, another drawer item): pause and remember
70      // the position. This happens in onPause rather than onStop because on recent Android
71      // versions the VideoView's surface, and with it the MediaPlayer, is already released
72      // by the time onStop runs, so the position would read as 0.
73      val videoView = binding.videoView
74      if (isPrepared) {
75          // Keep playing through a rotation; stay paused when the user really left the screen.
76          viewModel.savePlayback(
77              positionMs = videoView.currentPosition,
78              resumePlaying = videoView.isPlaying && requireActivity().isChangingConfigurations,
79          )
80      }
81      videoView.pause()
82  }
83
84  override fun onStop() {
85      super.onStop()
86      // The VideoView releases its player when its surface goes away; it prepares again (and
87      // calls onPrepared again) when the screen comes back.
88      isPrepared = false
89  }
```

- **Why `onPause` and not `onStop`?** On recent Android versions the VideoView's surface, and with it the MediaPlayer, is already released by the time `onStop` runs, so the position would read as 0.
- **`resumePlaying`** is true only if the video was playing *and* the activity is being recreated for a configuration change (`isChangingConfigurations`, i.e. a rotation). Leaving the screen for real always ends in "paused".

When the video is prepared again (after a rotation or a return to the screen), it jumps back to the saved position:

video/VideoFragment.kt lines 102–118

```
102 private fun onVideoPrepared() {
103     isPrepared = true
104     binding.videoLoading.isVisible = false
105     binding.videoView.seekTo(viewModel.positionMs)
106     if (viewModel.resumePlaying) {
107         binding.videoView.start()
108     } else {
109         // Briefly show the controls, so the play button is easy to find.
110         showControlsBriefly()
111     }
112 }
113
114 private fun onVideoCompleted() {
115     viewModel.onPlaybackCompleted()
116     // VideoView hides the controls at the end; show them again so replay is one tap away.
117     showControlsBriefly()
118 }
```

What the user does	What happens
Rotates while the video plays	Position saved, `resumePlaying = true` → new view seeks and **keeps playing**
Opens another drawer item, or presses Home	Position saved, `resumePlaying = false` → on return it waits, **paused**, at the same moment
Lets the video finish	`onPlaybackCompleted()` → next visit starts from 0; the controls appear for an easy replay

7.3 Player events, errors and the drawer

video/VideoFragment.kt lines 120–151

```
120  /**
121   * Shows the controls for MediaController's default few seconds. They are not pinned with
122   * show(0): the controller is a separate, focusable window, and while it is up the rest of
123   * the screen (toolbar, hamburger) cannot get keyboard focus. Tapping the video shows them
124   * again, and with TalkBack on MediaController ignores the timeout and keeps them visible.
125   */
126  private fun showControlsBriefly() {
127      mediaController?.show()
128  }
129
130  /** Shows a friendly message instead of the system "Can't play this video" dialog. */
131  private fun onVideoError(what: Int, extra: Int): Boolean {
132      Log.e(TAG, "VideoView error: what=$what extra=$extra")
133      binding.videoLoading.isVisible = false
134      binding.videoError.isVisible = true
135      return true // true = handled, so the system dialog is not shown
136  }
137
138  private fun showLoading() {
139      binding.videoError.isVisible = false
140      binding.videoLoading.isVisible = true
141  }
142
143  /**
144   * MediaController draws in its own window above the activity, so an opening drawer would
145   * slide *under* the controls. Hide them as soon as the drawer starts to move.
146   */
147  private val hideControlsWhileDrawerMoves = object : DrawerLayout.SimpleDrawerListener() {
148      override fun onDrawerStateChanged(newState: Int) {
149          if (newState != DrawerLayout.STATE_IDLE) mediaController?.hide()
150      }
151  }
```

- **Errors.** If the video cannot be played, `onVideoError` shows a friendly message and returns `true`, which tells VideoView "handled". Otherwise the system would show its own "Can't play this video" dialog.
- **Controls and the drawer.** `MediaController` draws in its own small window that floats above the activity, so an opening drawer would slide *under* the controls. The fragment therefore listens to the drawer and hides the controls as soon as it starts to move.
- **Controls timeout.** The controls are shown with MediaController's default timeout rather than pinned, because while that separate window is visible, the rest of the screen cannot receive keyboard focus.

Everything is cleaned up when the view goes away:

video/VideoFragment.kt lines 91–100

```kotlin
override fun onDestroyView() {
    drawerLayout?.removeDrawerListener(hideControlsWhileDrawerMoves)
    drawerLayout = null
    // The controller is shown in its own window; hide it so that window cannot leak.
    mediaController?.hide()
    mediaController = null
    binding.videoView.stopPlayback() // releases the MediaPlayer
    super.onDestroyView()
    _binding = null
}
```

PUB/SUB & MESSAGE-QUEUE VIEW

- The MediaPlayer inside VideoView reports **events** (prepared, error, completed) as messages posted to the main thread's queue; the listeners are their subscribers.
- Preparing is **asynchronous request/reply** again: "open this video" goes out, `onPrepared` comes back later. Only then is `seekTo` allowed.
- Returning `true` from `onVideoError` **acknowledges** the error message: it has been consumed, so the default handler (the system dialog) never sees it.
- The `DrawerLayout` has **two subscribers** at once while this screen is open: `MainActivity` 's listener (toggles the Back callback) and this fragment's `hideControlsWhileDrawerMoves` . One drawer event, fanned out to both. The fragment subscribes in `onViewCreated` and unsubscribes in `onDestroyView` , the matching pair.
- The saved position is another **store-and-forward** message: the screen that leaves writes it, the screen that comes back reads it.

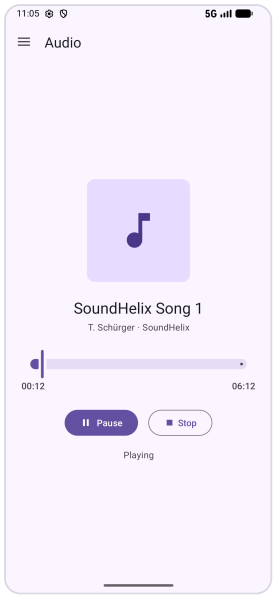
COMMON MISTAKE

Forgetting `stopPlayback()` or `mediaController.hide()` in `onDestroyView` . The first leaks the decoder and audio output; the second leaks the controller's window (Android logs a "window leaked" error).

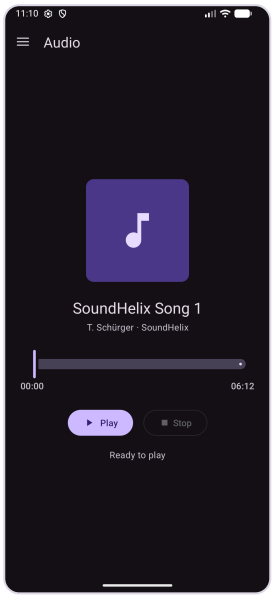
PART 8

Feature D: Audio

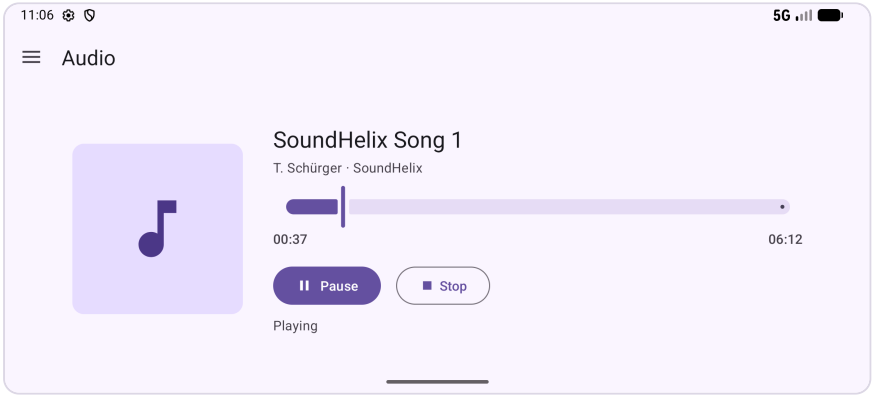
The brief: "Play one audio within app (media player)." The app plays *SoundHelix Song 1*, bundled as `res/raw/sample_audio.mp3`, with `android.media.MediaPlayer` used directly.



Playing



Dark theme



Landscape layout: artwork beside the controls

8.1 MediaPlayer is a state machine

A `MediaPlayer` is always in exactly one **state**, and each method is allowed only in certain states. Calling, say, `start()` before the player is prepared throws an exception or reports an error. So the order of calls matters:

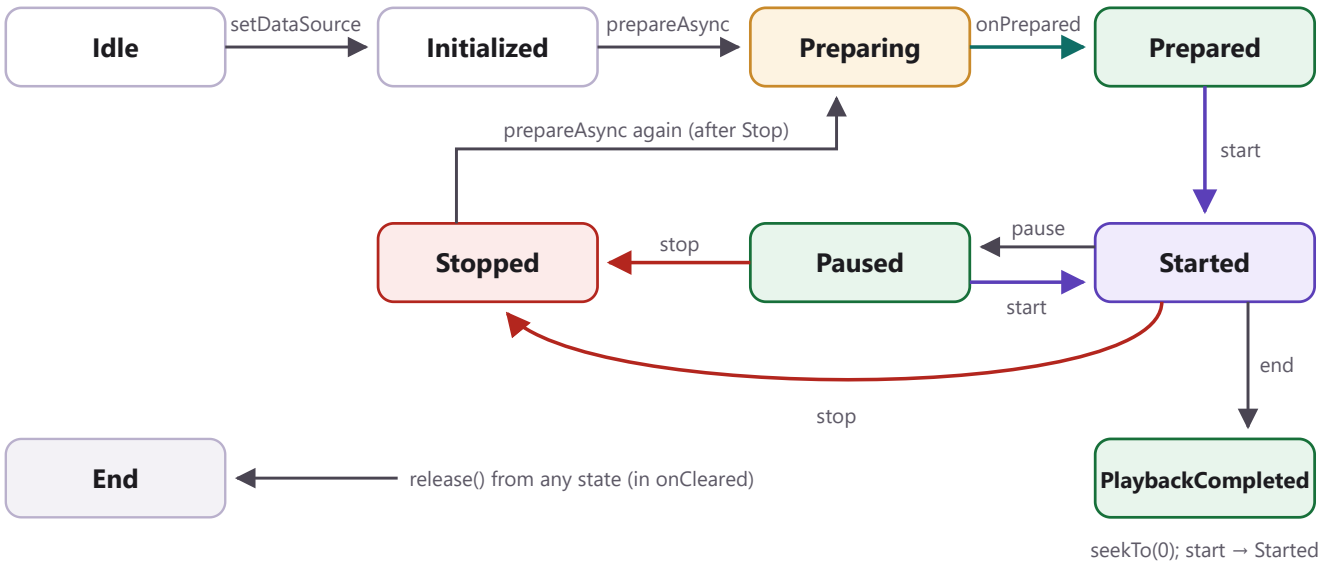


Figure 8.1 — The states this app moves through. Green states count as "prepared" for the UI.

The class comment in the code describes the same machine:

audio/AudioPlayerViewModel.kt lines 35–43

```
35  * MediaPlayer is a state machine; calling a method in the wrong state throws. This class moves
36  * through it like this:
37  * ```
38  * Idle --setDataSource--> Initialized --prepareAsync--> Preparing --(onPrepared)--> Prepared
39  * Prepared / Paused / PlaybackCompleted --start--> Started --pause--> Paused
40  * Started / Paused --stop--> Stopped --prepareAsync--> Preparing --> Prepared ...
41  * Started --(end of track)--> PlaybackCompleted
42  * any state --release--> End
43  * ```
```

- `prepareAsync()` instead of `prepare()`: preparing reads and decodes the start of the file and can take a moment, so it must not block the main thread.
- After `stop()` the player must be **prepared again** before it can play.
- `release()` frees the decoder and the audio output. Forgetting it leaks them.

8.2 The player lives in the ViewModel

audio/AudioPlayerViewModel.kt lines 45–70, 140–144

```
45  class AudioPlayerViewModel(application: Application) : AndroidViewModel(application) {
46
47      private val _uiState = MutableStateFlow(AudioUiState())
48      val uiState: StateFlow<AudioUiState> = _uiState.asStateFlow()
49
50      private val player = MediaPlayer()
51      private var progressJob: Job? = null
52
53      init {
54          // Idle -> Initialized. Audio attributes must be set before preparing.
55          player.setAudioAttributes(
56              AudioAttributes.Builder()
57                  .setUsage(AudioAttributes.USAGE_MEDIA)
58                  .setContentType(AudioAttributes.CONTENT_TYPE_MUSIC)
```

```
59                  .build(),
60          )
61          application.resources.openRawResourceFd(R.raw.sample_audio).use { file ->
62              player.setDataSource(file)
63          }
64          player.setOnPreparedListener { onPrepared() }
65          player.setOnCompletionListener { onCompleted() }
66          player.setOnErrorListener { _, what, extra -> onError(what, extra) }
67          // Initialized -> Preparing. prepareAsync() returns at once, so the UI never waits; the
68          // player calls onPrepared (on the main thread) when it is ready to play.
69          player.prepareAsync()
70      }
    ⋮
140     /** Called once the screen is gone for good: free the decoder and audio resources. */
141     override fun onCleared() {
142         stopProgressUpdates()
143         player.release() // any state -> End
144     }
```

- **Why in the ViewModel?** On rotation the fragment and its views are recreated, but the ViewModel (and so the player) stays. The music **does not stop** when you rotate the phone.
- `AudioAttributes` with `USAGE_MEDIA` and `CONTENT_TYPE_MUSIC` tell Android this is music, so it plays on the media volume. Attributes must be set before preparing.
- `openRawResourceFd(R.raw.sample_audio).use { … }` opens the bundled MP3, hands it to the player, and closes the file handle again (the player keeps its own).
- `onCleared()` runs only when the screen is gone for good. It stops the progress updates and releases the player.

8.3 State flows up, commands flow down

Everything the screen shows is one immutable object:

audio/AudioPlayerViewModel.kt lines 19–26

```
19  /** Everything the Audio screen shows. */
20  data class AudioUiState(
21      val isPrepared: Boolean = false,
22      val isPlaying: Boolean = false,
23      val positionMs: Int = 0,
24      val durationMs: Int = 0,
25      val hasError: Boolean = false,
26  )
```

The screen sends **commands**; the ViewModel changes the player and publishes a new state:

audio/AudioPlayerViewModel.kt lines 72–104

```
72  fun togglePlayPause() {
73      if (_uiState.value.isPlaying) pause() else play()
74  }
75
76  fun play() {
77      if (!_uiState.value.isPrepared || _uiState.value.isPlaying) return
78      player.start()
79      _uiState.update { it.copy(isPlaying = true) }
80      startProgressUpdates()
81  }
```

```
82
83  fun pause() {
84      if (!_uiState.value.isPlaying) return
85      player.pause()
86      stopProgressUpdates()
87      _uiState.update { it.copy(isPlaying = false, positionMs = player.currentPosition) }
88  }
89
90  /** Stops and rewinds. A stopped MediaPlayer must be prepared again before it can play. */
91  fun stop() {
92      if (!_uiState.value.isPrepared) return
93      player.stop()
94      stopProgressUpdates()
95      _uiState.update { it.copy(isPrepared = false, isPlaying = false, positionMs = 0) }
96      player.prepareAsync()
97  }
98
99  /** Moves playback to [positionMs] (from the seek slider). */
100 fun seekTo(positionMs: Int) {
101     if (!_uiState.value.isPrepared) return
102     player.seekTo(positionMs)
103     _uiState.update { it.copy(positionMs = positionMs) }
104 }
```

The player answers with **events**, which also become new states:

audio/AudioPlayerViewModel.kt lines 106–122

```
106 private fun onPrepared() {
107     _uiState.update { it.copy(isPrepared = true, durationMs = player.duration, hasError = false) }
108 }
109
110 /** End of the track: back to the start, ready to play again. */
111 private fun onCompleted() {
112     stopProgressUpdates()
113     player.seekTo(0)
114     _uiState.update { it.copy(isPlaying = false, positionMs = 0) }
115 }
116
117 private fun onError(what: Int, extra: Int): Boolean {
118     Log.e(TAG, "MediaPlayer error: what=$what extra=$extra")
119     stopProgressUpdates()
120     _uiState.update { it.copy(isPrepared = false, isPlaying = false, hasError = true) }
121     return true // handled, so onCompletion is not called as well
122 }
```

A MediaPlayer has no "position changed" event, so while playing, a coroutine **asks** for the position five times a second and publishes it:

audio/AudioPlayerViewModel.kt lines 124–138

```kotlin
/** While playing, publish the position a few times per second for the slider and timer. */
private fun startProgressUpdates() {
    progressJob?.cancel()
    progressJob = viewModelScope.launch {
        while (isActive) {
            _uiState.update { it.copy(positionMs = player.currentPosition) }
            delay(PROGRESS_INTERVAL_MS)
        }
    }
}

private fun stopProgressUpdates() {
    progressJob?.cancel()
    progressJob = null
}
```

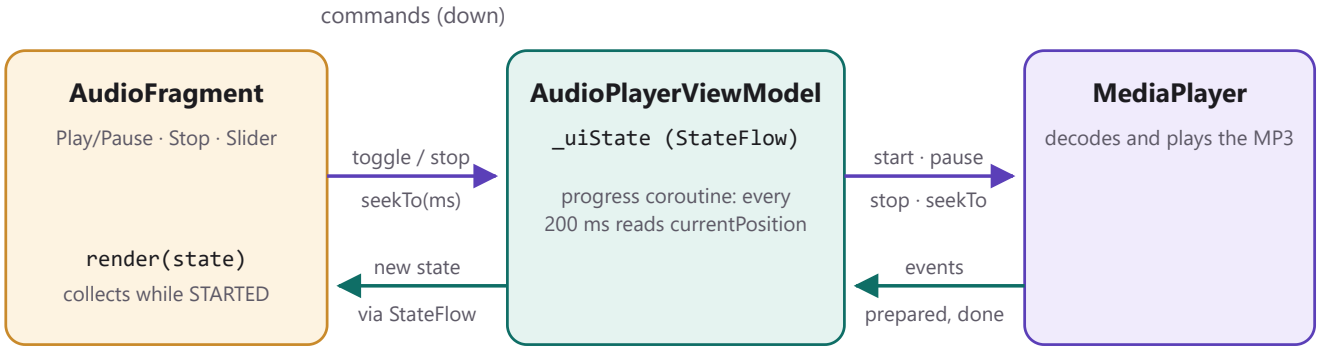


Figure 8.2 — Commands go down as direct calls; state comes back up through one StateFlow.

PUB/SUB & MESSAGE-QUEUE VIEW

- **Commands are point-to-point**: the fragment calls one method on one ViewModel (`togglePlayPause()`). **State is published**: the ViewModel does not know who draws it.
- `uiState` is a **retained, conflated topic**. After a rotation the new fragment gets the current state instantly. If the fragment is stopped, it simply misses intermediate positions and receives the latest one when it returns: only "now" matters.
- The progress coroutine is a **periodic producer** (polling). Each `delay(200)` is a delayed message in the main thread's queue (section 3.3), so the loop never blocks the screen. It is cancelled on pause, stop, end of track and `onCleared()` .
- `_uiState.update { it.copy(...) }` replaces the state in one atomic step, so two updates can never mix halves of each other.
- Returning `true` from `onError` **consumes** the error, so MediaPlayer does not also deliver `onCompletion` .

8.4 The screen: buttons, slider and times

audio/AudioFragment.kt — lines 30–36, 55–74

```
30  override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
31      super.onViewCreated(view, savedInstanceState)
32      _binding = FragmentAudioBinding.bind(view)
33
34      binding.playPauseButton.setOnClickListener { viewModel.togglePlayPause() }
35      binding.stopButton.setOnClickListener { viewModel.stop() }
36      setUpSeekSlider()
    ⋮
55  private fun setUpSeekSlider() {
56      val slider = binding.seekSlider
57      slider.setLabelFormatter { value -> formatMmSs(value.toLong()) }
58      slider.addOnChangeListener { _, value, fromUser ->
59          if (!fromUser) return@addOnChangeListener
60          binding.elapsedTime.text = formatMmSs(value.toLong())
61          // Keyboard and accessibility changes have no touch phase, so seek right away.
62          if (!isUserSeeking) viewModel.seekTo(value.toInt())
63      }
64      slider.addOnSliderTouchListener(object : Slider.OnSliderTouchListener {
65          override fun onStartTrackingTouch(slider: Slider) {
66              isUserSeeking = true
67          }
68
69          override fun onStopTrackingTouch(slider: Slider) {
70              isUserSeeking = false
71              viewModel.seekTo(slider.value.toInt())
72          }
73      })
74  }
```

The slider has one subtle problem: while your finger drags it, progress updates keep arriving five times a second and would yank the thumb back. So `isUserSeeking` is set while a finger is on the slider; during that time the screen **ignores** incoming positions for the slider and only seeks when the finger lifts. Keyboard and accessibility changes have no "finger lifted" moment, so they seek at once.

audio/AudioFragment.kt — lines 76–103

```
76  private fun render(state: AudioUiState) {
77      binding.playPauseButton.isEnabled = state.isPrepared
78      binding.playPauseButton.setText(if (state.isPlaying) R.string.audio_pause else R.string.audio_play)
79      binding.playPauseButton.setIconResource(
80          if (state.isPlaying) R.drawable.ic_pause_24 else R.drawable.ic_play_24,
81      )
82      binding.stopButton.isEnabled = state.isPrepared && (state.isPlaying || state.positionMs > 0)
83
84      // A Slider needs valueTo > valueFrom and a value inside that range.
85      val durationMs = state.durationMs.coerceAtLeast(1)
86      binding.seekSlider.isEnabled = state.isPrepared
87      binding.seekSlider.valueTo = durationMs.toFloat()
88      if (!isUserSeeking) {
89          binding.seekSlider.value = state.positionMs.coerceIn(0, durationMs).toFloat()
90          binding.elapsedTime.text = formatMmSs(state.positionMs.toLong())
91      }
92      binding.totalTime.text = formatMmSs(state.durationMs.toLong())
93
94      binding.audioStatus.setText(
95          when {
96              state.hasError -> R.string.audio_error
```

```
            !state.isPrepared -> R.string.audio_preparing
            state.isPlaying -> R.string.audio_playing
            state.positionMs > 0 -> R.string.audio_paused
            else -> R.string.audio_ready
        },
    )
}
```

- Play/Pause is disabled until prepared; its text and icon switch between Play and Pause.
- Stop is enabled only when there is something to stop or rewind.
- A Material `Slider` crashes if `valueTo` is not above `valueFrom`, so the duration is at least 1 ms before the track is prepared.
- The times are formatted as mm:ss by `formatMmSs` (section 9.1); the status line shows Preparing / Ready / Playing / Paused / error.

8.5 Audio and video never play together

Recall from section 4.2 that NavigationUI **keeps** the Audio screen's ViewModel (and its player) alive while you look at another drawer item. Without extra code, the music would continue behind the video. The fix:

audio/AudioFragment.kt lines 46–53

```
override fun onStop() {
    super.onStop()
    // Leaving the screen (another drawer item, Home, ...) pauses the music, so it never plays
    // on behind the video. NavigationUI keeps this screen's ViewModel (and its player) alive
    // while another drawer item is shown, so without this the audio would keep going.
    // A rotation is not "leaving": playback continues through it.
    if (!requireActivity().isChangingConfigurations) viewModel.pause()
}
```

The Video screen, for its part, pauses in `onPause` and never starts on its own when you come back (it resumes automatically only after a rotation, section 7.2). So the two players can never be heard at the same time.

What the user does	Audio	Video
Rotates the phone	Keeps playing (same player)	Keeps playing (position restored)
Switches to the other drawer item	Paused in `onStop`	Paused in `onPause`
Presses Home	Paused	Paused
Comes back	Waits, paused, at the same position	Waits, paused, at the same position

REMEMBER

`isChangingConfigurations` is the one-line difference between "the user rotated the phone" (keep playing) and "the user left the screen" (pause). Both features use it.

PART 9

Helpers, resources and build setup

9.1 Utility functions

`util/TimeFormat.kt` holds the two formatters used on screen:

util/TimeFormat.kt lines 7–29

```kotlin
private const val MILLIS_PER_SECOND = 1_000L
private const val SECONDS_PER_MINUTE = 60L

/**
 * Formats a playback position or duration as mm:ss, e.g. 65 000 ms -> "01:05".
 * Negative values count as zero; durations of an hour or more keep counting minutes ("75:00").
 */
fun formatMmSs(millis: Long): String {
    val totalSeconds = millis.coerceAtLeast(0L) / MILLIS_PER_SECOND
    val minutes = totalSeconds / SECONDS_PER_MINUTE
    val seconds = totalSeconds % SECONDS_PER_MINUTE
    return String.format(Locale.ROOT, "%02d:%02d", minutes, seconds)
}

/**
 * Formats a wall-clock time including seconds, following the user's 12/24-hour setting
 * (for example "9:41:07 PM" or "21:41:07"). Seconds make live updates easy to see.
 */
fun Context.formatTimeOfDay(millis: Long): String {
    val skeleton = if (DateFormat.is24HourFormat(this)) "Hms" else "hms"
    val pattern = DateFormat.getBestDateTimePattern(Locale.getDefault(), skeleton)
    return DateFormat.format(pattern, millis).toString()
}
```

Function	Example	Used by
`formatMmSs(millis)`	65 000 ms → `"01:05"`; negative → `"00:00"`; 75 min → `"75:00"`	Audio slider label and times
`Context.formatTimeOfDay(millis)`	`"10:56:42 PM"` or `"22:56:42"`, following the phone's 12/24-hour setting	Battery "Last update", received-message time

`Locale.ROOT` makes mm:ss always use Western digits, whatever the phone's language. Seconds are included in the clock time so that live updates are easy to see. The other helper, `util/WindowInsetsExt.kt`, was explained in section 4.6.

9.2 Resources: strings, layouts, themes

Strings

No text is written directly in layouts or code: everything is in `res/values/strings.xml`. This lets Android translate the app and lets lint check it. Some strings take **arguments**, and one is a **plural**, so "1 broadcast received" and "2 broadcasts received" are both correct English:

res/values/strings.xml lines 29, 52–55, 66

```
29  <string name="battery_percent">%1$d%%</string>
    ⋮
52  <plurals name="received_meta">
53      <item quantity="one">at %1$s · %2$d broadcast received</item>
54      <item quantity="other">at %1$s · %2$d broadcasts received</item>
55  </plurals>
    ⋮
66  <string name="zoom_level">%1$.1fx</string>
```

`%1$d%%` means "the first argument as a whole number, followed by a percent sign"; `%1$.1fx` means "the first argument with one decimal, followed by x" (e.g. `2.5x`).

Layouts

Layout file	What it is
`activity_main.xml`	DrawerLayout + toolbar + NavHostFragment + NavigationView (4.1)
`nav_header.xml`	The drawer's header: "CSE 489 · Assignment 2"
`fragment_broadcast.xml`, `item_spinner.xml`	Screen 1: label, Spinner inside an outlined card, description, Proceed; one Spinner row
`activity_broadcast_input.xml`	Activity 2: toolbar + `<include>` of `content_custom_input` and `content_battery_status`, both hidden until the mode is known
`activity_custom_receiver.xml`	Activity 3: toolbar + `<include>` of `content_custom_receiver` and `content_battery_nothing`
`fragment_image_scale.xml`	ZoomableImageView, loading ring, error + Retry, zoom label, hint
`fragment_video.xml`	Black frame with VideoView, loading ring and error text; title and credit
`fragment_audio.xml`	Artwork, title, slider, times, Play/Pause and Stop, status
`layout-land/content_battery_status.xml`, `layout-land/fragment_audio.xml`	Landscape versions: side-by-side instead of stacked

Android picks `layout-land/` automatically when the phone is sideways, and `values-land/dimens.xml` overrides a few sizes there. The landscape layouts use **the same view IDs** as the portrait ones, so a single ViewBinding class serves both.

res/values-land/dimens.xml lines 2–8

```
2  <resources>
3      <!-- Landscape phones are short: the battery ring shrinks to leave room for its details. -->
4      <dimen name="battery_indicator_size">140dp</dimen>
5      <!-- The audio artwork sits beside the controls in landscape, so it can stay large. -->
6      <dimen name="audio_artwork_size">180dp</dimen>
7      <dimen name="audio_artwork_icon_size">80dp</dimen>
8  </resources>
```

Theme, colours, icons, media

- **Theme**: `Theme.Material3.DayNight.NoActionBar`. DayNight switches between light and dark with the system setting; layouts use theme attributes (`?attr/colorSurface`, …) instead of fixed colours, so both

modes look right. *NoActionBar* because each screen has its own `MaterialToolbar`.

- **Colours**: only two fixed ones, black and white for the video frame, which is black in both themes.
- **Drawables**: vector icons (play, pause, stop, music note, info), the launcher icon and `bg_overlay_pill.xml`, the rounded background behind the zoom label and hint.
- **Raw**: `sample_video.mp4` (Sintel trailer, 4.4 MB) and `sample_audio.mp3` (SoundHelix Song 1, 8.9 MB).
- Touch targets are at least 48 dp, and sizes live in `dimens.xml` rather than in the layouts.

9.3 The manifest

AndroidManifest.xml lines 5–46

```
5  <!-- Needed by Coil to download the picture on the Image Scale screen. -->
6  <uses-permission android:name="android.permission.INTERNET" />
7
8  <!--
9      enableOnBackInvokedCallback opts Android 13-15 into predictive back as well (it is the
10     default from Android 16 with targetSdk 36+), so Back behaves the same everywhere.
11 -->
12 <application
13     android:allowBackup="true"
14     android:dataExtractionRules="@xml/data_extraction_rules"
15     android:enableOnBackInvokedCallback="true"
16     android:fullBackupContent="@xml/backup_rules"
17     android:icon="@mipmap/ic_launcher"
18     android:label="@string/app_name"
19     android:roundIcon="@mipmap/ic_launcher_round"
20     android:supportsRtl="true"
21     android:theme="@style/Theme.LabAssignment2_CSE489"
22     tools:targetApi="33">
23
24     <!-- The only exported activity: the launcher entry point. -->
25     <activity
26         android:name=".MainActivity"
27         android:exported="true">
28         <intent-filter>
29             <action android:name="android.intent.action.MAIN" />
30
31             <category android:name="android.intent.category.LAUNCHER" />
32         </intent-filter>
33     </activity>
34
35     <!--
36         Activity 2 and Activity 3 of the broadcast flow. Only this app starts them, with
37         explicit Intents, so they are not exported. adjustResize makes the system send IME
38         insets, which Activity 2 uses to keep its text field above the keyboard.
39     -->
40     <activity
41         android:name=".broadcast.BroadcastInputActivity"
42         android:exported="false"
43         android:windowSoftInputMode="adjustResize" />
44     <activity
45         android:name=".broadcast.CustomReceiverActivity"
46         android:exported="false" />
```

- `INTERNET`: needed for the Image Scale download.
- `enableOnBackInvokedCallback="true"`: predictive back on Android 13–15 too (4.4).

- Only `MainActivity` is `exported="true"`; Activities 2 and 3 are `false`, so no other app can open them (5.11).
- `adjustResize` on Activity 2 makes the system send keyboard (IME) insets, which keep the text field above the keyboard (4.6).
- No `<receiver>` elements: both receivers are context-registered in code (3.2).

PUB/SUB & MESSAGE-QUEUE VIEW

The `<intent-filter>` with `MAIN` + `LAUNCHER` is a **static subscription** written in the manifest. When you tap the app icon, the launcher sends an Intent with that action and category, and the system routes it to the one activity whose filter matches. `android:exported` is the **access-control list** for these point-to-point messages: who outside the app may send them.

9.4 Gradle build and dependencies

app/build.gradle.kts lines 6–43

```
6  android {
7      namespace = "com.example.labassignment2_cse489"
8      compileSdk {
9          version = release(37)
10     }
11
12     defaultConfig {
13         applicationId = "com.example.labassignment2_cse489"
14         minSdk = 24
15         targetSdk = 37
16         versionCode = 1
17         versionName = "1.0"
18
19         testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
20     }
21
22     buildTypes {
23         release {
24             optimization {
25                 enable = true
26                 packageScope = setOf("androidx.**", "kotlin.**", "kotlinx.**")
27             }
28         }
29     }
30     compileOptions {
31         sourceCompatibility = JavaVersion.VERSION_11
32         targetCompatibility = JavaVersion.VERSION_11
33     }
34     buildFeatures {
35         viewBinding = true
36         // BuildConfig.APPLICATION_ID is used to build the custom broadcast action name.
37         buildConfig = true
38     }
39     testOptions {
40         // Espresso is more reliable with system animations switched off during tests.
41         animationsDisabled = true
42     }
43 }
```

- **SDK levels**: `minSdk 24` (runs on Android 7.0 and newer), `targetSdk 37` and `compileSdk 37` (built for, and against, the newest Android). The target level switches on the modern rules described in this document: edge-to-edge, predictive back, receiver export flags.
- `viewBinding = true` generates the binding classes; `buildConfig = true` generates `BuildConfig.APPLICATION_ID`, used to build the custom action name.
- `optimization { enable = true }` turns on R8 shrinking for release builds.
- `animationsDisabled = true` switches system animations off during instrumented tests.

app/build.gradle.kts lines 45–69

```
45  dependencies {
46      implementation(libs.androidx.core.ktx)
47      implementation(libs.androidx.appcompat)
48      implementation(libs.material)
49      implementation(libs.androidx.constraintlayout)
50      implementation(libs.androidx.drawerlayout)
51      implementation(libs.androidx.activity)
52      implementation(libs.androidx.fragment)
53      implementation(libs.androidx.lifecycle.runtime)
54      implementation(libs.androidx.lifecycle.viewmodel)
55      implementation(libs.androidx.lifecycle.viewmodel.savedstate)
56      implementation(libs.androidx.lifecycle.livedata)
57      implementation(libs.androidx.navigation.fragment)
58      implementation(libs.androidx.navigation.ui)
59      implementation(libs.coil)
60      implementation(libs.coil.network.okhttp)
61
62      testImplementation(libs.junit)
63
64      androidTestImplementation(libs.androidx.junit)
65      androidTestImplementation(libs.androidx.test.runner)
66      androidTestImplementation(libs.androidx.test.rules)
67      androidTestImplementation(libs.androidx.espresso.core)
68      androidTestImplementation(libs.androidx.espresso.contrib)
69  }
```

Library versions are not written here but in a **version catalog**, `gradle/libs.versions.toml` (for example `lifecycle = "2.11.0"`, `navigation = "2.10.2"`, `coil = "3.6.3"`). `libs.androidx.navigation.ui` in the build file refers to an entry there. The comment in the catalog explains why the plain artifacts are used instead of the old `-ktx` ones: the Kotlin extensions (`by viewModels()`, `viewModelScope`, `setupWithNavController`) now live in the main artifacts.

The root `build.gradle.kts` declares (but does not apply) a newer Kotlin Gradle plugin: AGP 9 compiles Kotlin itself, and Coil 3.6 needs a newer Kotlin compiler than the one AGP bundles.

PART 10

Tests

The project has **44 automated tests**: 26 unit tests that run on your computer in seconds, and 18 instrumented tests that run on an emulator or phone and click through the real app.

Running them (Windows PowerShell, in the project folder)

```
.\gradlew.bat testDebugUnitTest           # 26 JVM unit tests
.\gradlew.bat connectedDebugAndroidTest   # 18 instrumented tests (needs an emulator/phone)
.\gradlew.bat lintDebug                   # Android lint
```

10.1 Unit tests

Unit tests check **pure logic**: code that needs no screen. That is exactly why the maths was moved out of the Android classes into `BatteryStatus`, `BroadcastMode`, `ZoomMath` and `TimeFormat`.

File (in `app/src/test/`)	Tests	What is checked
`broadcast/BatteryStatusTest.kt`	9	Percentage for scale 100 and other scales, rounding down, 0 and 100, scale 0 or negative → `null`, missing level → `null`, level above scale clamped; every `EXTRA_STATUS` value and unexpected ones
`broadcast/BroadcastModeTest.kt`	5	Spinner position ↔ mode, out-of-range positions, name round-trip, unknown/missing/wrong-case names, and the exact action and extra names
`image/ZoomMathTest.kt`	8	Zoom clamped to 1x–5x, fit for wide/tall/small pictures, centring, closing gaps at the start and at the end, no change when covered
`util/TimeFormatTest.kt`	4	Padding, dropping partial seconds, tracks over an hour, negative values

Two examples. The first protects against the division by zero discussed in 5.10; the second checks the topic name both sides of the broadcast rely on:

test/broadcast/BatteryStatusTest.kt lines 34–45

```
34  @Test
35  fun percentOf_zeroOrNegativeScale_returnsNull() {
36      // Guards against dividing by zero when the broadcast has no usable scale.
37      assertNull(BatteryStatus.percentOf(level = 50, scale = 0))
38      assertNull(BatteryStatus.percentOf(level = 50, scale = -1))
39  }
40
41  @Test
42  fun percentOf_missingLevel_returnsNull() {
43      // getIntExtra() returns the default -1 when EXTRA_LEVEL is missing.
44      assertNull(BatteryStatus.percentOf(level = -1, scale = 100))
45  }
```

test/broadcast/BroadcastModeTest.kt lines 39–49

```kotlin
@Test
fun customBroadcast_actionAndExtra_arePrefixedWithApplicationId() {
    assertEquals(
        "com.example.labassignment2_cse489.action.CUSTOM_MESSAGE",
        CustomMessageReceiver.ACTION_CUSTOM_MESSAGE,
    )
    assertEquals(
        "com.example.labassignment2_cse489.extra.MESSAGE",
        CustomMessageReceiver.EXTRA_MESSAGE,
    )
}
```

10.2 Instrumented tests

These use **Espresso**, which finds views and performs clicks and typing like a user would.

File (in `app/src/androidTest/`)	Tests	What is checked
`DrawerNavigationTest.kt`	7	The four labels in order; starting on Broadcast Receiver; opening every screen (title, hamburger, highlighted item); Back closes the drawer without leaving the app
`broadcast/BroadcastFlowTest.kt`	5	The two Spinner items; the whole custom flow with the button and with the keyboard's Send key; blank-input error; the battery flow and Activity 3 doing nothing
`image/ZoomableImageViewTest.kt`	6	Synthetic two-finger pinches through the real `ScaleGestureDetector` (zoom anchored at the fingers, limited to 5x and 1x), panning only when zoomed, edge clamping, double-tap zoom and reset

The most important test runs the full pub/sub flow end to end: "Hello CSE489" typed in Activity 2 must come out of the receiver in Activity 3.

androidTest/broadcast/BroadcastFlowTest.kt lines 51–69

```kotlin
@Test
fun customFlow_textTypedInActivity2_isReceivedByReceiverInActivity3() {
    selectBroadcastType(CUSTOM)
    onView(withId(R.id.proceed_button)).perform(click())

    // Activity 2: type the message and send it.
    onView(withId(R.id.message_input)).perform(typeText(MESSAGE), closeSoftKeyboard())
    onView(withId(R.id.send_button)).perform(click())

    // Activity 3: CustomMessageReceiver is registered and shows the broadcast text.
    onView(withId(R.id.receiver_status))
        .check(matches(withText("CustomMessageReceiver is registered")))
    onView(withId(R.id.receiver_action))
        .check(matches(withText("Action: ${CustomMessageReceiver.ACTION_CUSTOM_MESSAGE}")))
    waitUntil {
        onView(withId(R.id.received_message))
            .check(matches(withText("Received via broadcast: $MESSAGE")))
    }
}
```

The pinch test proves the "point under the fingers stays under the fingers" rule from Figure 6.1:

androidTest/image/ZoomableImageViewTest.kt lines 52–66

```kotlin
@Test
fun pinchOut_zoomsInAroundTheFingers() {
    val focusX = 600f
    val focusY = 800f
    val imagePointUnderFingers = onMain { viewToImage(focusX, focusY) }

    pinch(focusX, focusY, startSpan = 1000f, endSpan = 1500f)

    val zoom = onMain { view.zoom }
    assertTrue("zoom should grow, was $zoom", zoom > 1.3f && zoom < 1.6f)
    // The picture point that was under the fingers is still under the fingers.
    val nowAt = onMain { imageToView(imagePointUnderFingers[0], imagePointUnderFingers[1]) }
    assertEquals(focusX, nowAt[0], PIXEL_TOLERANCE)
    assertEquals(focusY, nowAt[1], PIXEL_TOLERANCE)
}
```

`connectedDebugAndroidTest` uninstalls the app when it finishes, so run `installDebug` again before a demo.

10.3 Why tests must wait for messages

Espresso automatically waits until the app's **main-thread message queue is idle** before each check: no pending messages means the screen has finished updating. But a broadcast is not in that queue yet when `sendBroadcast()` returns. It is first routed through `system_server`, in another process (Figure 3.1), and only arrives in the app's queue a moment later. Espresso cannot see it coming, so the project adds a small helper that retries a check until it passes:

androidTest/TestUtils.kt lines 16–34

```kotlin
/**
 * Retries [assertion] until it passes or [timeoutMs] runs out.
 *
 * Espresso waits for the main thread to be idle, but not for things that happen outside the
 * app's process: a broadcast is delivered by the system a moment after sendBroadcast(), and a
 * drawer slides open frame by frame. Polling keeps such checks reliable.
 */
fun waitUntil(timeoutMs: Long = DEFAULT_TIMEOUT_MS, assertion: () -> Unit) {
    val deadline = SystemClock.uptimeMillis() + timeoutMs
    while (true) {
        try {
            assertion()
            return
        } catch (failure: Throwable) {
            if (SystemClock.uptimeMillis() > deadline) throw failure
            SystemClock.sleep(POLL_INTERVAL_MS)
        }
    }
}
```

PUB/SUB & MESSAGE-QUEUE VIEW

This is the testing face of **synchronization decoupling** (section 2.4). Because the publisher does not wait for delivery, the result becomes visible only *eventually*. A test of an asynchronous system must therefore **poll with a timeout** instead of checking once. The same helper is used for the drawer, which slides open frame by frame (each frame is another message).

PART 11

Summary and revision

11.1 One-page cheat sheet

Feature	Key code	How it works	Messaging idea
Drawer & navigation	`MainActivity.setUpNavigation`, `nav_graph.xml`, `drawer_menu.xml`	Menu IDs = destination IDs; NavigationUI navigates, sets the title, highlights	NavController publishes destination changes to the toolbar and drawer (fan-out)
Back	`closeDrawerOnBack`	Callback enabled only while the drawer is open	One consumer wins each Back event
Screen 1	`BroadcastFragment`, `BroadcastViewModel`, `BroadcastMode`	Spinner → ViewModel (SavedStateHandle) → LiveData → description; Proceed → explicit Intent	Unidirectional data flow; point-to-point message with a validated payload
Activity 2 · custom	`BroadcastInputActivity.sendMessage`	Trim, reject blank, put the text in the Intent to Activity 3	Validate before sending; reliable point-to-point channel
Activity 3 · custom	`CustomReceiverActivity`, `CustomMessageReceiver`, `CustomReceiverViewModel`	Create receiver → register in `onStart` → `sendBroadcast` → `onReceive` → show; unregister in `onStop`	Subscribe first, publish second (non-durable topic); producer-side de-duplication
Battery	`BatteryStatusLiveData`, `BatteryStatus`	Register in `onActive` (sticky value arrives at once), unregister in `onInactive`; level × 100 / scale	System topic with a retained message; reference-counted bridge into LiveData
Image Scale	`ImageScaleViewModel`, `ZoomableImageView`, `ZoomMath`	Coil at original size; Matrix scale/translate; zoom around the focal point, clamp 1x–5x, no gaps	Async request/reply; touch events as a consumed stream; detectors as stream processors
Video	`VideoFragment`, `VideoViewModel`	VideoView + MediaController; save position in `onPause`; resume only after rotation	Player events as messages; error acknowledged (`return true`); store-and-forward position
Audio	`AudioPlayerViewModel`, `AudioFragment`	Player in the ViewModel; StateFlow of `AudioUiState`; 200 ms progress loop; pause in `onStop`	Commands down, state up; retained conflated topic; periodic producer
Everywhere	Lifecycle callbacks, `repeatOnLifecycle`, main Looper	Subscribe and unsubscribe in matching callbacks; all UI work on the main thread	Lifecycle = subscription manager; main thread = queue with one consumer

THE PROJECT IN ONE MINUTE

"One activity hosts four fragments through the Navigation component and a drawer. Every screen keeps its state in a ViewModel, so rotation loses nothing. The Broadcast Receiver feature uses Android's publish/subscribe system: the custom path passes the text point-to-point in an Intent, then Activity 3 registers its own receiver and broadcasts the text to it, because a normal broadcast is not kept for receivers that register later. The battery path subscribes to the system's sticky `ACTION_BATTERY_CHANGED` through a LiveData that listens only while the screen is visible. Image Scale pinch-zooms a Coil-loaded photo with `ScaleGestureDetector` and a Matrix. Video and Audio use `MediaPlayer`, survive rotation, and pause when you leave, so they never play together. Underneath, every callback is a message handled by the main thread's Looper."

11.2 Likely viva questions

Pub/Sub and message queues

How is an Android broadcast an example of publish/subscribe?

`sendBroadcast()` publishes an Intent whose *action* is the topic. Receivers subscribe with `registerReceiver(receiver, IntentFilter(action))`. The ActivityManagerService in `system_server` is the broker: it matches filters and delivers a copy to every matching receiver. The publisher never knows who the subscribers are.

Where is there a message queue in this app?

On the main thread: the `Looper` takes messages from the `MessageQueue` one at a time. Touches, lifecycle callbacks, `onReceive`, `onPrepared`, animation frames and coroutine timers all arrive that way. It is a queue with exactly one consumer. The system also keeps its own `BroadcastQueue` of broadcasts waiting for delivery.

What is the difference between a message queue and pub/sub?

In a queue each message goes to exactly one consumer (work is shared). In pub/sub every subscriber of the topic gets its own copy (events are announced).

Why doesn't Activity 2 send the custom broadcast?

A normal broadcast is non-durable: it reaches only receivers registered at that moment. The receiver is created in Activity 3, so it would be lost. The text travels in the Intent instead, and Activity 3 broadcasts it after registering: subscribe first, publish second.

What is a sticky broadcast? What is that called in messaging?

The system keeps the last copy and `registerReceiver()` returns it immediately. In messaging it is a *retained* or *last-value* message (MQTT's retain flag). `ACTION_BATTERY_CHANGED` is sticky, so the battery level appears at once.

Which kinds of decoupling do broadcasts give?

Space (publisher and receiver share only the action name) and synchronization (`sendBroadcast` returns at once, delivery is asynchronous), but not time (late subscribers miss normal broadcasts).

What delivery guarantee does a normal broadcast give?

At-most-once: delivered to the current receivers, never retried, dropped if nobody listens.

How do you avoid sending the custom broadcast twice when the screen rotates?

`CustomReceiverViewModel.initialBroadcastSent`, stored in the `SavedStateHandle`. Only the first `onStart` publishes. This is producer-side de-duplication.

When exactly is the battery receiver registered?

Only while Activity 2 is visible. `BatteryStatusLiveData` registers in `onActive()` (first observer STARTED) and unregisters in `onInactive()`. It subscribes upstream only while it has downstream subscribers.

Is LiveData publish/subscribe?

It is the Observer pattern: in-process, no broker. It behaves like a retained, conflated topic (a new observer gets the latest value), and `observe(owner)` ties the subscription to a lifecycle.

Who may send to your custom receiver, and who may read your broadcast?

`RECEIVER_NOT_EXPORTED` means only this app and the system can send to the receiver.
`setPackage(packageName)` means only this app can receive the broadcast. The battery broadcast is protected: only the system may send it.

Why must `onReceive()` be quick?

It runs on the main thread, the single consumer of the UI queue. While it runs, touches and drawing wait; if it takes too long, Android reports the app as not responding (ANR).

Android

Why fragments instead of four activities?

The toolbar and drawer stay in one activity and only the content changes. Navigation handles titles, highlighting and the back stack, and fragments are lighter to switch.

Difference between `onDestroyView` and `onDestroy` in a fragment?

`onDestroyView` destroys only the views (for example when you switch drawer items); the fragment may live on. That is why `_binding = null` happens there.

How does the drawer know which screen to open?

The menu item IDs equal the destination IDs in `nav_graph.xml`; `navView.setupWithNavController` navigates to the matching ID.

Why does every screen show ☰ and not a back arrow?

`AppBarConfiguration(navView.menu, drawerLayout)` makes all four top-level destinations.

How does Back close the drawer first?

An `OnBackPressedCallback` that is enabled only while the drawer is open (`onDrawerOpened` / `onDrawerClosed`). `onBackPressed()` is not called on Android 16+ with targetSdk 36+.

Context-registered or manifest receiver? Why?

Context-registered, both. The brief wants the custom receiver created in Activity 3, and `ACTION_BATTERY_CHANGED` can only be received by a receiver registered in code.

How does pinch zoom keep the point under the fingers?

`Matrix.postScale(s, s, focusX, focusY)` scales around the focal point, which therefore does not move. The factor is clamped to 1x–5x first.

Why `Size.ORIGINAL` in Coil?

Otherwise Coil decodes at the view's size, and zooming in shows a blurry, upscaled bitmap.

Why is the MediaPlayer in a ViewModel?

The ViewModel survives rotation, so the music continues; it is released in `onCleared()`.

What is `prepareAsync()` for?

Preparing can take time. Doing it asynchronously keeps the UI responsive; `onPrepared` says when `start()` is allowed.

Why does the Video screen save its position in `onPause`?

By `onStop` the VideoView may already have released its player with its surface, and the position would read 0.

How do audio and video never play together?

Audio pauses in `onStop` unless rotating; video pauses in `onPause` and only resumes by itself after a rotation.

What survives the process being killed?

Values in a `SavedStateHandle`: the Spinner choice, Activity 3's received message and "already sent" flag, and the video position. A plain ViewModel field does not.

11.3 Glossary

Term	Meaning
Action	The string in an Intent that says what kind of message it is; the topic of a broadcast.
Binder	Android's mechanism for one process to call another, used to reach the system.
Broker	The middleman that stores and routes messages. For broadcasts: ActivityManagerService.
BroadcastReceiver	A class whose `onReceive()` is called for matching broadcasts: a subscriber.
Conflation	Keeping only the newest undelivered value and dropping older ones.
Context-registered receiver	A receiver registered in code; it exists only until it is unregistered.
De-duplication	Making sure the same message is not processed or sent twice.
Explicit Intent	An Intent that names its exact target class: a point-to-point message.
Fan-out	One published message delivered as a copy to many subscribers.
Fragment	A piece of screen with its own lifecycle, hosted in an activity.
Idempotent	Doing it twice has the same effect as doing it once.
IntentFilter	Describes which Intents a receiver wants: the subscription filter.
LiveData	A lifecycle-aware observable value holder.
Looper / MessageQueue	The main thread's event loop and its time-sorted queue of messages.
Matrix	A table of numbers that maps picture pixels to screen positions (scale + translation).
Observer pattern	A subject notifies a list of observers directly; pub/sub without a broker.
Payload	The data part of a message; for an Intent, its extras.
Point-to-point	Messaging where each message goes to exactly one receiver (a queue).
Protected broadcast	A broadcast that only the system is allowed to send.
Publish/subscribe	Messaging where publishers send to a topic and every subscriber gets a copy.
Retained message	The last message of a topic, kept for new subscribers. Android: sticky broadcast.
SavedStateHandle	Key-value state in a ViewModel that also survives process death.

Term	Meaning
ScaleGestureDetector	Turns two-finger touch events into scale factor + focal point.
StateFlow	A coroutine stream that always holds a current value.
Topic	A named channel in pub/sub.
ViewBinding	Generated classes with typed fields for every view in a layout.
ViewModel	Holds a screen's state and survives rotation.

11.4 How to run and try every feature

Requirements: Android Studio with the Android SDK Platform 37, JDK 17 or newer (Android Studio's bundled JBR works), and internet for the first Gradle sync and the Image Scale screen.

Build and install from the command line (Windows PowerShell)

```
# Only needed if Gradle cannot find a JDK:
$env:JAVA_HOME = 'C:\Program Files\Android\Android Studio\jbr'

.\gradlew.bat assembleDebug        # builds app\build\outputs\apk\debug\app-debug.apk
.\gradlew.bat installDebug         # installs it on the connected emulator/phone
```

Or in Android Studio: *File › Open* the project folder, wait for the sync, pick a device, press Run ▶.

Feature	Try this
Drawer	Tap ☰ or swipe from the left edge; pick each item (title and highlight change). Open the drawer and press Back: it closes, the app stays.
Custom broadcast	Keep "Custom broadcast receiver" → Proceed. Tap Send with an empty field (error). Type `Hello CSE489` → Send. Activity 3 shows "registered", the action, and "Received via broadcast: Hello CSE489" with a Toast. Tap Broadcast again (counter goes up). Rotate: same text, nothing re-sent. Logcat: `adb logcat -s CustomMessageReceiver`
Battery	Choose the battery option → Proceed: the values appear at once (sticky). Change the battery in the emulator's *Extended controls › Battery*, or with `adb shell dumpsys battery set level 42`, `adb shell dumpsys battery set status 2` (2 = charging, 3 = discharging, 5 = full) and `adb shell dumpsys battery reset`. Next → "Nothing to do here".
Image Scale	Wait for the photo; pinch (emulator: hold Ctrl and drag); drag while zoomed (no gaps); double-tap (2.5x) and again (1x); rotate (zoom kept). Offline with a fresh install: error, then Retry.
Video	Tap the video for the controls, press ▶, seek. Rotate while playing (continues). Switch drawer item and back (paused at the same moment).
Audio	Play/Pause, Stop (rewinds), drag or tap the slider. Rotate while playing (keeps playing). Switch to Video or press Home (music pauses).

Credits: video *Sintel* trailer © Blender Foundation (CC BY 3.0); audio *SoundHelix Song 1* by T. Schürger; photo by Alexey Topolyanskiy on Unsplash, served by Lorem Picsum; icons from Material Icons (Apache 2.0). Theory reference: P. Th. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, "The Many Faces of Publish/Subscribe", *ACM Computing Surveys* 35(2), 2003.